

Privacy-Preserving Fingerprint Authentication

Core Problem Statement

Fingerprint authentication on a cloud server requires the server to *compare* a biometric query against a database, yet biometric data is among the most sensitive personal information that exists. This creates a fundamental tension between security and scalability. Two existing solutions each solve half the problem: (1) Blind-Touch uses homomorphic encryption (HE) to keep templates encrypted on the server but suffers from large ciphertext expansion and a structural scale ceiling - requiring a 117 MB Galois key, 855 KB per 512 users, and becoming impractical beyond roughly 5,000 users; (2) Fuzzy-labeled PSI (FLPSI) is communication-efficient, scales linearly to millions of records without HE overhead, but operates only on raw binary strings under Hamming distance and has no mechanism to incorporate the deep-learning feature extractors that drive modern biometric perceptual accuracy.

The open problem this paper solves is getting *both* CNN-level perceptual accuracy and FLPSI-level communication efficiency and scalability in a single privacy-preserving system.

1 Introduction

Biometric authentication offers strong usability and security guarantees for cloud and web services, but centralized storage of biometric templates creates extreme privacy risks. Unlike passwords, compromised fingerprints cannot be revoked or replaced. The canonical real-world risk materialized in the 2015 United States Office of Personnel Management (OPM) data breach, where 5.6 million fingerprint records were stolen. Privacy-preserving protocols are thus essential to allow authentication without ever exposing the raw or plaintext biometric data to the server.

Existing solutions to this problem remain insufficient. **Homomorphic Encryption (HE)-based** methods, such as Blind-Touch, successfully obscure the data from the server and achieve high accuracy. However, they suffer from expensive ciphertext expansion, large rotation keys, and a structural scaling bottleneck due to ciphertext slot-packing limits. **PSI-based** methods (Uzun et al., Bui-Cong) offer cheap communication and massive scalability but assume inputs are already raw bit-strings; they lack a robust biometric feature extractor to handle the natural analog variations (rotation, translation, noise) of real fingerprint captures.

Our key insight is that Blind-Touch’s accuracy comes entirely from its convolutional neural network (CNN) feature extractor, *not* from its HE matching layer. Once the CNN produces a compact floating-point embedding, the matching problem effectively reduces to an approximate nearest-neighbour search in Hamming space - exactly the primitive that Fuzzy-Labeled Private Set Intersection (FLPSI) solves at massive scale. Decoupling the perceptual feature extraction (run locally in plaintext on the client) from the privacy-preserving matching (run between client and server via PSI) is this paper’s central contribution.

Specifically, we make the following contributions:

- **(C1)** A novel system design combining Siamese CNN feature extraction with scalable FLPSI matching, eliminating the homomorphic encryption layer entirely.

- **(C2)** A binary quantization bridge (Super-Bit) that successfully maps the CNN’s floating-point embeddings into the fixed-length Hamming space required by PSI, without sacrificing perceptual accuracy.
- **(C3)** A formal semi-honest security proof for the composite system.
- **(C4)** A full prototype implementation and evaluation, with measurements to $N = 5 \times 10^4$ and an analytic cost model beyond it.

2 Background

2.1 Notation

Throughout this paper, let λ denote the security parameter. All cryptographic algorithms are assumed to run in probabilistic polynomial time (PPT) with respect to λ .

We denote a finite field of prime order q by $\mathbb{F}_q$, where all arithmetic operations are performed modulo q . For a positive integer n , the notation

$$[n] = \{1, 2, \dots, n\}$$

represents the set containing the first n positive integers.

Vectors are represented using bold lowercase letters, such as $\mathbf{u}$ and $\mathbf{v}$, whereas matrices are represented using bold uppercase letters, such as $\mathbf{A}$ and $\mathbf{B}$. Unless otherwise stated, all vectors are assumed to be column vectors.

Binary strings of length d are represented as elements of the set

$$\{0, 1\}^d.$$

For two binary vectors

$$\mathbf{x}, \mathbf{y} \in \{0, 1\}^d,$$

their Hamming distance is denoted by

$$d_H(\mathbf{x}, \mathbf{y}),$$

which corresponds to the number of bit positions at which the two vectors differ.

The notation

$$x \stackrel{\$}{\leftarrow} S$$

indicates that the value x is sampled uniformly at random from the finite set S . Similarly,

$$x \leftarrow \mathcal{A}$$

denotes that x is produced by algorithm $\mathcal{A}$.

Whenever a protocol fails to produce a valid output, the special symbol

$$\perp$$

is returned.

2.2 Homomorphic Encryption and CKKS

Homomorphic encryption (HE) permits computation directly on ciphertexts: an evaluator produces an encrypted result that, once decrypted, equals the result of the same computation on the plaintext, without ever learning the underlying values. The *CKKS* scheme of Cheon, Kim, Kim, and Song (2017) is the variant suited to biometric matching because it supports approximate arithmetic on real numbers rather than exact integers: it packs a vector of reals into the slots of a single polynomial ciphertext, so one homomorphic operation acts on all packed values in SIMD fashion. Each homomorphic multiplication consumes one level of a finite *modulus chain*, so the multiplicative depth of an evaluable circuit is fixed in advance by the chosen parameters, and choosing a larger depth or more slots directly inflates ciphertext size and key material. Blind-Touch builds on exactly this scheme, and it is also the source of its scalability cost: a large Galois key and per-user ciphertext that grows linearly with the database.

2.3 Siamese Neural Networks for Fingerprint Matching

Blind-Touch [4] frames fingerprint authentication as a similarity-matching problem using a *Siamese* network: two weight-sharing sub-networks process a pair of fingerprints, and a small head decides whether they belong to the same identity. We reuse this architecture. The shared feature extractor is five convolutional blocks of increasing width ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ filters); each block applies a 3×3 convolution, batch normalisation, a swish activation, and 2×2 max-pooling, reducing a 224×224 grayscale fingerprint to a $7 \times 7 \times 512$ feature map that is flattened and L2-normalised. The matching head takes the two feature vectors, subtracts them, passes the difference through a 16-unit dense layer (*FC-16*), squares the activations, and applies a single sigmoid unit to output a match probability in $[0, 1]$; above the acceptance threshold (we used 0.99) the pair is declared a match. We reproduced this pipeline end-to-end, training to 0.9958 validation accuracy on SOCOFing, with a genuine query authenticating at score 0.998441.

The FC-16 layer with its square activation is the compact, learned representation on which the match decision turns, and it is also the intended handoff point to our system: a 16-dimensional floating-point embedding is exactly the right size to quantise into a short Hamming-space code and match with FLPSI instead of HE. This keeps Blind-Touch’s perceptual accuracy while discarding the costly encrypted-matching layer.

2.4 Oblivious Transfer

Oblivious Transfer (OT) is one of the fundamental primitives in modern cryptography and serves as a building block for a wide range of secure two-party computation protocols. Informally, OT enables a sender to transfer one of several messages to a receiver without learning which message was requested, while simultaneously preventing the receiver from obtaining any information about the remaining messages.

Definition 1 (1-out-of-2 Oblivious Transfer). *A protocol securely realizes a 1-out-of-2 Oblivious Transfer functionality if it satisfies the following properties.*

1. *The sender inputs two messages*

$$(m_0, m_1).$$

2. *The receiver inputs a choice bit*

$$b \in \{0, 1\}.$$

3. *The receiver learns only the selected message*

$$m_b.$$

4. *The sender obtains no information regarding the receiver's selection bit b .*

5. *The receiver obtains no information regarding the unselected message*

$$m_{1-b}.$$

The security of Oblivious Transfer relies on satisfying two complementary privacy guarantees.

Receiver Privacy. The sender should remain computationally unable to distinguish which message has been requested by the receiver. Formally, for any probabilistic polynomial-time sender, the protocol transcript should be computationally indistinguishable regardless of whether the receiver chooses $b = 0$ or $b = 1$.

Sender Privacy. The receiver should obtain no information regarding the unselected message beyond what can be inferred from the selected message. Consequently, learning m_b should reveal no computationally useful information about m_{1-b} .

The ideal functionality of a 1-out-of-2 Oblivious Transfer can therefore be represented as

$$\mathcal{F}_{OT} : ((m_0, m_1), b) \rightarrow m_b,$$

where the sender learns nothing about b , and the receiver learns nothing beyond the selected message.

OT offers strong privacy guarantees but performing an independent public-key protocol for each transfer is computationally expensive for large-scale secure computation. Modern protocols, e.g., Private Set Intersection and Secure Multi-Party Computation, often require millions of Oblivious Transfers in a single protocol execution. Thus, doing a public-key operation for each transfer would be a huge computational overhead.

2.5 Correlated Oblivious Transfer

Standard Oblivious Transfer enables the sender to send two arbitrary messages. However, many secure computation protocols do not require totally independent messages. Instead, the messages transmitted are often known to satisfy a predetermined algebraic relation. This observation yields a more efficient primitive called Correlated Oblivious Transfer (COT).

In Correlated OT, the sender no longer chooses two independent messages. Instead, a global correlation value

$$\Delta \in \mathbb{F}_q$$

is fixed before the protocol begins. For every transfer, the sender selects a random value

$$x_i \xleftarrow{\$} \mathbb{F}_q,$$

and constructs the correlated message pair

$$(x_i, x_i + \Delta).$$

The receiver possesses a choice bit

$$b_i \in \{0, 1\},$$

and receives

$$y_i = \begin{cases} x_i, & b_i = 0, \\ x_i + \Delta, & b_i = 1. \end{cases}$$

Consequently, the receiver learns only one of the two correlated messages, while the sender remains unaware of the receiver's selection.

Unlike standard OT, the sender does not explicitly transmit two arbitrary messages. Instead, every transfer is completely determined by the random value x_i and the global correlation Δ , which significantly reduces both communication and computational complexity.

2.6 Vector Oblivious Linear Evaluation

While Correlated Oblivious Transfer generates pairs of correlated field elements, many secure computation protocols require correlations over vectors rather than individual values. Vector Oblivious Linear Evaluation (VOLE) extends the idea of Correlated OT by producing correlated vectors that satisfy a linear relation over a finite field. These correlations form the basis of several modern cryptographic protocols, including Oblivious Pseudorandom Functions (OPRFs), Private Set Intersection (PSI), and secure multiparty computation.

Let

$$\Delta \in \mathbb{F}_q$$

be a secret global correlation value known only to the sender, where $\mathbb{F}_q$ denotes a finite field of prime order q . Consider a vector of length n .

The receiver samples two random vectors

$$\mathbf{u} = (u_1, u_2, \dots, u_n) \in \mathbb{F}_q^n,$$

and

$$\mathbf{w} = (w_1, w_2, \dots, w_n) \in \mathbb{F}_q^n.$$

Using the secret correlation value Δ , the sender computes the vector

$$\mathbf{v} = (v_1, v_2, \dots, v_n),$$

where each element satisfies

$$v_i = u_i \Delta + w_i, \quad 1 \leq i \leq n.$$

Equivalently, the vector relation can be expressed as

$$\mathbf{v} = \Delta \mathbf{u} + \mathbf{w}.$$

The resulting vectors satisfy a linear correlation determined by the secret value Δ . From the sender's perspective, only the correlation value

$$\Delta$$

and the correlated vector

$$\mathbf{v}$$

are available. The sender has no knowledge of the receiver's vectors

$$\mathbf{u} \quad \text{or} \quad \mathbf{w}.$$

Conversely, the receiver knows only

$$(\mathbf{u}, \mathbf{w}),$$

while the correlation value

$$\Delta$$

remains computationally hidden.

Consequently, neither party can independently reconstruct the complete linear relation, while both parties collectively hold additive shares satisfying

$$\mathbf{v} = \Delta \mathbf{u} + \mathbf{w}.$$

Unlike conventional Oblivious Transfer, which transfers independent messages, VOLE directly generates correlated randomness satisfying a linear equation over a finite field. This property allows subsequent cryptographic protocols to perform arithmetic operations using only local computations without requiring additional interaction between the participating parties.

Another important advantage of VOLE is that the correlation value Δ is reused across the entire vector. As a result, a single secret value generates a large number of correlated field elements, significantly improving communication efficiency compared to executing independent oblivious transfers.

2.7 Silent VOLE Extension

The Vector Oblivious Linear Evaluation (VOLE) primitive introduced in the previous section efficiently generates correlated randomness between two parties. However, many modern cryptographic protocols, including Oblivious Pseudorandom Functions (OPRFs), Private Set Intersection (PSI), and Secure Multi-Party Computation (MPC), require millions of VOLE correlations during a single execution. Generating each correlation independently would require repeatedly executing expensive cryptographic operations, making the protocol impractical for large-scale applications.

Silent VOLE Extension overcomes this limitation by expanding a relatively small number of *base VOLE correlations* into a substantially larger collection of correlated vectors using only lightweight local computations and a small amount of communication. Consequently, the expensive public-key operations are performed only during the generation of the base correlations, while the remaining VOLE instances are produced through efficient symmetric-key techniques.

Assume that the parties initially possess

$$m$$

base VOLE correlations,

$$\{(\Delta, u_1, v_1), (\Delta, u_2, v_2), \dots, (\Delta, u_m, v_m)\},$$

where

$$v_i = \Delta u_i + w_i,$$

and

$$m \ll n.$$

The objective of Silent VOLE is to derive

$$n$$

correlated vectors satisfying the same VOLE relation,

$$n \gg m,$$

without executing additional public-key cryptographic protocols.

The key idea underlying Silent VOLE is to combine a *Pseudorandom Generator (PRG)* with a *linear error-correcting code*. Instead of independently generating every VOLE correlation, both parties first expand a short amount of shared randomness into long pseudorandom vectors using a cryptographically secure PRG. These expanded vectors are subsequently processed using a linear code based on the Learning Parity with Noise (LPN) assumption. The linear encoding preserves the algebraic structure of the original VOLE correlations, allowing the parties to obtain a significantly larger collection of valid correlated vectors while performing only inexpensive linear algebra operations.

Conceptually, the expansion process can be viewed as

Base VOLE $\rightarrow$ PRG Expansion $\rightarrow$ Linear Encoding $\rightarrow$ Millions of VOLE Correlations.

Since the majority of the computation consists only of pseudorandom generation and matrix-vector operations, Silent VOLE achieves substantially lower communication and computational complexity than executing independent VOLE instances.

The protocol naturally separates into two phases.

Offline Phase The offline phase is executed before the protocol inputs are known. During this phase, the parties generate a small number of base VOLE correlations together with the random seeds required by the pseudorandom generator. These seeds are locally expanded into long pseudorandom vectors, which are then processed using the linear encoding procedure to generate a large collection of correlated randomness. Since this phase is independent of the participants' private inputs, it may be performed during system initialization or whenever computational resources are available.

The output of the offline phase is a large collection of correlated vectors satisfying the VOLE functionality

$$v = \Delta u + w,$$

while revealing neither the receiver's vectors nor the sender's secret correlation value.

Online Phase When the actual protocol inputs become available, the parties reuse the previously generated correlations instead of executing additional VOLE protocols. Only lightweight local computations together with a small amount of communication are required to adapt the preprocessed correlations to the protocol execution. Consequently, the online phase incurs very low latency compared with the expensive preprocessing stage.

The security of Silent VOLE follows from the security of the underlying base VOLE protocol, the pseudorandomness of the employed PRG, and the hardness of the Learning Parity with Noise (LPN) problem underlying the linear encoding scheme. An adversary capable of distinguishing the generated correlations from uniformly random correlations would therefore break one of these underlying cryptographic assumptions.

Silent VOLE is particularly well suited for protocols requiring a large amount of correlated randomness during preprocessing. In Bui and Cong's [3] authentication framework, these expanded VOLE correlations are later utilized by the Vector Ring Oblivious Linear Evaluation protocol, which converts the vector correlations into secure polynomial correlations required by the Threshold Labelled Private Set Intersection protocol.

2.8 Polynomial Interpolation

Several cryptographic protocols employed throughout this work represent polynomials by evaluating them at a collection of distinct points rather than directly manipulating their coefficients. Recovering the original polynomial from these evaluation values is achieved through *polynomial interpolation*.

Suppose a polynomial

$$f(x)$$

has degree at most d . A fundamental result from algebra states that such a polynomial is uniquely determined by its values at any $d + 1$ distinct points. Therefore, if the evaluations

$$(x_1, y_1), (x_2, y_2), \dots, (x_{d+1}, y_{d+1}),$$

are known, where

$$y_i = f(x_i),$$

the original polynomial can be reconstructed uniquely.

One of the most commonly used interpolation techniques is the *Lagrange interpolation formula*, which reconstructs the polynomial as

$$f(x) = \sum_{i=1}^{d+1} y_i L_i(x),$$

where

$$L_i(x) = \prod_{\substack{1 \leq j \leq d+1 \\ j \neq i}} \frac{x - x_j}{x_i - x_j}$$

denotes the i^{th} Lagrange basis polynomial.

In the FLPSI authentication framework of Bui and Cong [3], polynomial interpolation is used during the Vector Ring-OLE protocol to reconstruct polynomial products from their evaluations over a common set of field elements. Instead of performing coefficient-wise polynomial multiplication, polynomials are first evaluated at several distinct points, arithmetic is performed on these evaluation values, and the resulting polynomial is subsequently recovered using interpolation. This evaluation-interpolation strategy substantially improves computational efficiency and forms the foundation of the Vector Ring-OLE construction presented in the next section.

2.9 Vector Ring Oblivious Linear Evaluation

The Threshold Labelled Private Set Intersection (TLPSI) protocol requires repeated secure evaluations of polynomial products while preserving the privacy of the communicating parties. Although the Vector Oblivious Linear Evaluation (VOLE) primitive introduced in the previous section efficiently generates correlated randomness over vectors, it does not directly support the polynomial computations required for polynomial interpolation and Shamir secret sharing.

To address this limitation, the protocol employs the *Vector Ring Oblivious Linear Evaluation* (Vector Ring-VOLE) primitive. Instead of operating directly on coefficient vectors, Vector Ring-VOLE represents each participant's input as a polynomial over a finite field and performs secure polynomial multiplication through evaluations at a common set of interpolation points. Consequently, polynomial operations are transformed into element-wise operations over evaluation vectors, allowing the protocol to reuse the efficient Vector VOLE primitive while avoiding explicit polynomial multiplication.

Unlike conventional Ring-OLE protocols defined over quotient polynomial rings, Vector Ring-VOLE operates directly over the polynomial ring without polynomial reduction. Furthermore, the protocol is specifically designed for the asymmetric setting encountered in TLPSI, where the receiver owns a single polynomial while the sender possesses a large collection of polynomials. This enables multiple polynomial products to be generated simultaneously, substantially reducing both computation and communication compared to executing independent Ring-OLE instances.

2.9.1 Mathematical Representation

Let

$$R = \mathbb{F}[x]$$

denote the polynomial ring over a finite field $\mathbb{F}$.

Assume that the receiver owns a polynomial

$$u_0(x) \in R,$$

whose degree is at most d , while the sender possesses a collection of m polynomials

$$\{v_i(x)\}_{i=1}^m,$$

each having degree at most $2d$.

For every sender polynomial, the Vector Ring-VOLE functionality samples a uniformly random polynomial

$$u_i(x),$$

of degree at most d and produces the polynomial

$$z_i(x) = u_0(x) u_i(x) + v_i(x), \quad i = 1, \dots, m.$$

Since

$$\deg(u_0) \leq d \quad \text{and} \quad \deg(u_i) \leq d,$$

their product satisfies

$$\deg(u_0(x)u_i(x)) \leq 2d.$$

Therefore,

$$\deg(z_i) \leq 2d,$$

and every resulting polynomial is uniquely determined by its evaluations at any collection of $2d + 1$ distinct field elements.

Throughout the protocol, let

$$X = \{x_1, x_2, \dots, x_{2d+1}\} \subset \mathbb{F}$$

denote a publicly known set of pairwise distinct interpolation points.

For any polynomial

$$p(x) \in R,$$

its evaluation vector over the interpolation set is denoted by

$$p(X) = (p(x_1), p(x_2), \dots, p(x_{2d+1})).$$

For simplicity, the evaluation vectors of

$$u_i(x), v_i(x), z_i(x)$$

will be denoted respectively by

$$U_i, V_i, Z_i.$$

This distinction between lowercase polynomial notation and uppercase evaluation-vector notation is maintained throughout the remainder of this chapter.

2.9.2 Ideal Functionality

The Vector Ring-VOLE functionality is executed between a sender and a receiver.

- The receiver provides a polynomial

$$u_0(x) \in R$$

of degree at most d .

- The sender provides a collection of polynomials

$$\{v_i(x)\}_{i=1}^m,$$

each having degree at most $2d$.

The functionality independently samples uniformly random degree- d polynomials

$$u_1(x), u_2(x), \dots, u_m(x),$$

and computes

$$z_i(x) = u_0(x) u_i(x) + v_i(x), \quad i = 1, \dots, m.$$

Finally,

- the sender receives

$$\{u_i(x)\}_{i=1}^m,$$

- the receiver receives

$$\{z_i(x)\}_{i=1}^m.$$

Neither participant obtains any additional information regarding the other participant's private polynomials beyond what is implied by the ideal functionality.

Unlike conventional Ring-OLE protocols, the same receiver polynomial

$$u_0(x)$$

is reused across all protocol instances. This property makes the generated correlations particularly well suited for batch polynomial evaluation in the Threshold Labelled Private Set Intersection protocol.

2.9.3 Evaluation Perspective

Direct multiplication of polynomial coefficient vectors is computationally expensive and unsuitable for efficient batch execution. Vector Ring-VOLE therefore performs polynomial multiplication using the evaluation-interpolation principle.

Every polynomial is first evaluated over the common interpolation set

$$X = \{x_1, x_2, \dots, x_{2d+1}\}.$$

For each interpolation point

$$x_j \in X,$$

the protocol computes

$$z_i(x_j) = u_0(x_j) u_i(x_j) + v_i(x_j).$$

Equivalently, using evaluation-vector notation,

$$Z_i = u_0(X) \odot U_i + V_i,$$

where

$$\odot$$

denotes component-wise multiplication.

Since every polynomial

$$z_i(x)$$

has degree at most $2d$, its evaluation vector contains exactly the information required for unique reconstruction. Applying Lagrange interpolation yields

$$z_i(x) = \sum_{j=1}^{2d+1} z_i(x_j) L_j(x),$$

where

$$L_j(x) = \prod_{\substack{1 \leq k \leq 2d+1 \\ k \neq j}} \frac{x - x_k}{x_j - x_k}$$

is the j -th Lagrange basis polynomial.

Consequently, secure polynomial multiplication is reduced to secure element-wise multiplication over evaluation vectors. This observation forms the foundation of the Vector Ring-VOLE construction and enables the protocol to leverage the efficient Vector VOLE primitive introduced in the previous section.

2.9.4 Offline (Preprocessing) Phase

The offline phase is responsible for generating random correlated data that can later be transformed into the Vector Ring-VOLE correlations required during the online execution. Since this preprocessing stage is independent of the participants' actual private inputs, it can be executed before either party provides its secret polynomial. Consequently, the computationally expensive cryptographic operations are moved entirely to the offline stage, allowing the online phase to consist primarily of lightweight finite-field operations.

The preprocessing procedure is built upon the Silent VOLE Extension protocol described in the previous section. Starting from a small number of base VOLE correlations, the protocol efficiently expands them into a large collection of correlated random vectors. These vectors serve as reusable resources that are later adapted to the participants' actual inputs during the online phase.

Let

$$X = \{x_1, x_2, \dots, x_{2d+1}\} \subset \mathbb{F}$$

denote the common interpolation set introduced previously.

Since the product of two degree- d polynomials has degree at most $2d$, exactly $2d+1$ distinct evaluation points are sufficient for unique polynomial reconstruction through interpolation.

Assume that the protocol must generate correlations for

$$m$$

sender polynomials.

Generation of Random Polynomials The receiver first samples a uniformly random polynomial

$$u'_0(x) \in R,$$

whose degree is at most d .

Independently, the sender samples a collection of uniformly random polynomials

$$v'_1(x), v'_2(x), \dots, v'_m(x),$$

each having degree at most $2d$.

These random polynomials are completely independent of the participants' actual protocol inputs. Their sole purpose is to establish reusable random correlations that can later be transformed into correlations corresponding to the desired secret polynomials.

Construction of Evaluation Vectors Instead of manipulating polynomial coefficients directly, every randomly generated polynomial is evaluated over the common interpolation set.

The receiver computes the evaluation vector

$$u'_0(X) = (u'_0(x_1), u'_0(x_2), \dots, u'_0(x_{2d+1})) .$$

Similarly, for every sender polynomial

$$v'_i(x),$$

the sender computes

$$V'_i = v'_i(X) = (v'_i(x_1), v'_i(x_2), \dots, v'_i(x_{2d+1})) .$$

Collecting all sender evaluation vectors produces the matrix

$$V' = \begin{bmatrix} V'_1 \\ V'_2 \\ \vdots \\ V'_m \end{bmatrix} ,$$

whose dimensions are

$$m \times (2d + 1).$$

Each row corresponds to one sender polynomial, whereas each column corresponds to one interpolation point.

Generation of Random VOLE Correlations Using the evaluation vectors constructed above, the parties execute the Silent VOLE Extension protocol.

For every sender polynomial, the protocol generates a random correlated evaluation vector

$$U'_i,$$

which may be viewed as the evaluation vector of a random degree- d polynomial obtained through polynomial interpolation.

Collecting all generated vectors yields

$$U' = \begin{bmatrix} U'_1 \\ U'_2 \\ \vdots \\ U'_m \end{bmatrix}.$$

For every row, the receiver obtains a correlated evaluation vector

$$Z'_i = u'_0(X) \odot U'_i + V'_i, \quad i = 1, \dots, m,$$

where

$$\odot$$

denotes component-wise multiplication.

Equivalently, the complete collection of correlations can be written compactly as

$$Z' = u'_0(X) \odot U' + V'.$$

These correlations are uniformly random because they are generated independently of the participants' actual secret inputs.

Interpretation of the Preprocessing Data At the conclusion of the preprocessing stage,

- the receiver possesses

$$u'_0(x), \quad Z',$$

- the sender possesses

$$U', \quad V'.$$

Although these values satisfy valid Vector VOLE correlations, they correspond only to randomly generated polynomials rather than the participants' actual private inputs. Consequently, they cannot yet be used for the Threshold Labelled Private Set Intersection protocol.

Instead, the preprocessing data acts as reusable correlated randomness. During the online phase, a small correction procedure transforms these random correlations into Vector Ring-VOLE correlations corresponding to the receiver's secret polynomial and the sender's actual polynomial collection. Since the expensive Silent VOLE Extension has already been completed offline, this transformation requires only lightweight finite-field operations together with a small amount of communication.

2.9.5 Online Phase

After completing the offline preprocessing stage, both parties possess a collection of random correlated evaluation vectors that are independent of their actual protocol inputs. Consequently, these correlations cannot yet be used for the Threshold Labelled Private Set Intersection protocol.

The objective of the online phase is to transform the random preprocessing correlations into valid Vector Ring-VOLE correlations corresponding to the participants' secret input polynomials. This transformation is commonly referred to as *correlation adjustment* (or *de-randomization*) and requires only a small amount of communication together with inexpensive finite-field operations. Importantly, no additional execution of the Silent VOLE Extension protocol is required.

Assume that the receiver's secret polynomial is

$$u_0(x),$$

while the sender holds the collection of secret polynomials

$$\{v_i(x)\}_{i=1}^m.$$

The preprocessing stage has already generated the random evaluation vectors satisfying

$$Z' = u'_0(X) \odot U' + V',$$

where

$$u'_0(X) = (u'_0(x_1), u'_0(x_2), \dots, u'_0(x_{2d+1}))$$

denotes the evaluation vector of the randomly generated receiver polynomial.

The goal of the online phase is to obtain correlations corresponding to

$$u_0(x)$$

without repeating the expensive preprocessing computation.

Correlation Adjustment The receiver first computes the correction polynomial

$$\Delta u(x) = u_0(x) - u'_0(x).$$

Since both polynomials have degree at most d , the correction polynomial also has degree at most d .

Rather than transmitting the coefficients of the correction polynomial, the receiver evaluates it over the common interpolation points

$$X = \{x_1, x_2, \dots, x_{2d+1}\},$$

obtaining the correction vector

$$\Delta U = \Delta u(X) = (\Delta u(x_1), \Delta u(x_2), \dots, \Delta u(x_{2d+1})).$$

Only this evaluation vector is transmitted to the sender. Consequently, the receiver never reveals the coefficients of its secret polynomial.

Using the received correction vector together with the preprocessing data, the sender adjusts the stored correlations so that they now correspond to the receiver's actual polynomial rather than the randomly generated preprocessing polynomial. This adjustment consists only of finite-field additions over the stored evaluation vectors and therefore incurs negligible computational overhead compared to the offline Silent VOLE Extension protocol.

After the adjustment, the resulting evaluation vectors satisfy the Vector Ring-VOLE relation

$$Z = u_0(X) \odot U + V,$$

where

$$U = \{U_i\}_{i=1}^m, \quad V = \{V_i\}_{i=1}^m,$$

represent the sender's evaluation vectors corresponding to the final protocol execution.

Thus, the random preprocessing correlations are converted into correlations corresponding to the participants' actual secret inputs without performing another VOLE protocol.

Polynomial Reconstruction Each evaluation vector

$$Z_i = (z_i(x_1), z_i(x_2), \dots, z_i(x_{2d+1}))$$

contains the evaluations of a polynomial whose degree is at most $2d$. Therefore, the receiver reconstructs the polynomial

$$z_i(x)$$

using Lagrange interpolation,

$$z_i(x) = \sum_{j=1}^{2d+1} z_i(x_j) L_j(x),$$

where

$$L_j(x) = \prod_{\substack{1 \leq k \leq 2d+1 \\ k \neq j}} \frac{x - x_k}{x_j - x_k}$$

is the j -th Lagrange basis polynomial.

The reconstructed polynomial satisfies

$$z_i(x) = u_0(x)u_i(x) + v_i(x),$$

which is precisely the polynomial required by the ideal Vector Ring-VOLE functionality.

2.9.6 Correctness

The correctness of the Vector Ring-VOLE protocol follows directly from the preprocessing stage together with the online correlation adjustment.

During preprocessing, the parties generate random evaluation vectors satisfying

$$Z' = u'_0(X) \odot U' + V'.$$

These vectors are independent of the participants' actual inputs and therefore can be generated entirely offline.

During the online phase, the receiver computes the correction polynomial

$$\Delta u(x) = u_0(x) - u'_0(x),$$

and communicates only its evaluations over the common interpolation points. The sender subsequently adjusts the stored preprocessing correlations so that the resulting evaluation vectors satisfy

$$Z = u_0(X) \odot U + V.$$

For every interpolation point

$$x_j \in X,$$

the corresponding coordinate satisfies

$$z_i(x_j) = u_0(x_j) u_i(x_j) + v_i(x_j).$$

Since

$$\deg(z_i) \leq 2d,$$

the polynomial is uniquely determined by its evaluations at the $2d + 1$ interpolation points. Applying Lagrange interpolation therefore reconstructs the unique polynomial

$$z_i(x) = u_0(x) u_i(x) + v_i(x),$$

which is exactly the output specified by the ideal Vector Ring-VOLE functionality.

Hence, the protocol produces the same result as direct polynomial multiplication while preserving the privacy guarantees established by the underlying VOLE correlations.

2.10 Security Analysis

The security of the Vector Ring-VOLE protocol follows directly from the security guarantees provided by the underlying Silent VOLE Extension protocol together with the correctness of the correlation adjustment performed during the online phase.

Throughout the execution, the receiver never reveals its secret polynomial

$$u_0(x).$$

Instead, the preprocessing stage is executed using a uniformly random polynomial

$$u'_0(x),$$

which is statistically independent of the receiver's actual input. Consequently, the correlated randomness generated during preprocessing leaks no information about the receiver's secret polynomial.

During the online phase, the receiver communicates only the evaluation vector of the correction polynomial

$$\Delta u(x) = u_0(x) - u'_0(x).$$

Since the preprocessing polynomial is chosen uniformly at random and is known only to the receiver, these correction values reveal no useful information about the coefficients of the receiver's actual polynomial beyond what is implied by the protocol output.

Similarly, the sender's private polynomial collection

$$\{v_i(x)\}_{i=1}^m$$

is never disclosed to the receiver. The receiver obtains only the correlated output polynomials

$$\{z_i(x)\}_{i=1}^m,$$

which are computationally indistinguishable from the outputs generated by the ideal Vector Ring-VOLE functionality.

Assuming the security of the Silent VOLE Extension protocol in the semi-honest adversarial model, the constructed Vector Ring-VOLE protocol securely realizes the ideal functionality while preserving the privacy of both participants’ inputs.

3 Problem Formulation

We model the privacy-preserving fingerprint authentication problem with three main entities: the user (client), the authentication server, and an offline key authority (for setup phases). The process is split into two phases: registration, where a user enrolls their biometric template, and authentication, where a probe fingerprint is queried against the server’s database.

We define three precise security goals for the system:

- **(G1) Biometric Privacy:** The server learns nothing about the query fingerprint or its distance to any registered template during authentication.
- **(G2) Template Privacy:** The client learns nothing about the server’s registered templates beyond the identity of the single matched record (if any).
- **(G3) Correctness:** Genuine fingerprint pairs authenticate successfully (subject to the false rejection rate), and imposter pairs are rejected (subject to the false acceptance rate).

Consistent with prior work, we operate in the semi-honest (honest-but-curious) threat model. Malicious adversaries, side-channel attacks, and presentation attacks (such as spoofing or liveness detection) are out of scope.

Existing state-of-the-art solutions address only subsets of these goals. Blind-Touch strictly achieves G1 and G3 but relies on Homomorphic Encryption, leading to poor scalability that prevents deployment for large-scale databases. Conversely, the FLPSI construction of Bui–Cong achieves G1, G2, and G3 with massive scalability, but it assumes the inputs are already binary strings, which is not the case for raw, noisy fingerprint images. NeuralPSI explicitly targets all three goals simultaneously while maintaining practical efficiency.

4 System Design: NeuralPSI

4.1 Architecture Overview

The NeuralPSI architecture is strictly partitioned into two phases to enforce privacy: client-side feature extraction and server-side matching.

Client Side: The client captures the raw fingerprint image and passes it through the Siamese CNN to extract the 16-dimensional floating-point embedding (FC-16). This continuous embedding is then passed through the quantisation bridge to produce a 128-bit binary string $\tilde{\mathbf{x}} \in \{0, 1\}^{128}$. This entire pipeline (Image $\rightarrow$ CNN $\rightarrow$ FC-16 $\rightarrow$ quantisation) runs locally in plaintext on the user’s own device.

Server Side: The authentication server simply stores a plaintext database of registered binary templates and their associated labels, $\{(\tilde{\mathbf{y}}_i, \ell_i)\}$.

During authentication, the client and server engage in an interactive FLPSI protocol. The client provides its binary query $\tilde{\mathbf{x}}$, and the server provides its database.

The core design principle of NeuralPSI is that the computationally heavy and perceptually difficult task—the CNN—runs entirely in plaintext on the client. No raw biometric data, and no floating-point neural embeddings, ever leave the device. The server only ever sees PRF-masked sub-samples of the binary code $\tilde{\mathbf{x}}$ during the PSI protocol, ensuring strict biometric privacy.

4.2 Feature Quantisation: From CNN Embeddings to Hamming Space

NeuralPSI replaces Blind-Touch’s homomorphic-encryption matching layer with FLPSI, and this swap creates a representation mismatch the rest of the system cannot paper over. The CNN feature extractor emits a 16-dimensional floating-point embedding $\mathbf{e} \in \mathbb{R}^{16}$ whose natural notion of similarity is cosine (equivalently Euclidean, after L2-normalisation); FLPSI, by contrast, matches fixed-length *binary* strings under *Hamming* distance with a sub-sampling threshold. Contribution C2 is the converter between them: a map from $\mathbf{e}$ to a binary code $\tilde{\mathbf{x}} \in \{0, 1\}^{128}$ under which two impressions of the same finger land at small Hamming distance and impressions of different fingers land far apart, in exactly the form the PSI protocol - and the security argument of §5 - can consume. A usable bridge must satisfy four constraints at once, and most off-the-shelf hashing schemes violate at least one: (i) *similarity preservation*, Hamming distance must track identity; (ii) *fixed binary format*, exactly $d = 128$ native Hamming bits; (iii) *an FLPSI-friendly error profile*, because the matcher sub-samples bit positions the per-bit errors should be near-i.i.d. and the bits balanced ($\approx 50/50$), or the protocol’s FPR/FNR analysis no longer holds; and (iv) *a public, cheap map*, published alongside the model (like batch-normalisation statistics) and costing a single matrix multiply at inference.//

Our quantiser is a fixed, public, *affine-then-sign* map - with no learned non-linearity - in three stages applied to the embedding $\mathbf{e}$. **(1) Centre:** $\mathbf{e}_0 = \mathbf{e} - \boldsymbol{\mu}$, where $\boldsymbol{\mu}$ is the population mean on a held-out calibration split; L2-normalised features occupy a spherical cap rather than the full sphere, so thresholding at the origin would yield badly unbalanced bits, and centring restores balance. **(2) Super-Bit projection:** $\mathbf{z} = \mathbf{P}\mathbf{e}_0$ with $\mathbf{P} \in \mathbb{R}^{128 \times 16}$ built as 8 stacked 16×16 Gaussian blocks, each orthonormalised within the block by Gram-Schmidt, all generated from a single *published seed*. **(3) Balanced sign:** $\tilde{x}_k = \mathbf{1}[z_k > \tau_k]$, where τ_k is the per-bit median on the calibration split, forcing each output bit to $\approx 50/50$ as constraint (iii) requires. At inference the three stages collapse to one matrix multiply and a threshold compare, $\tilde{\mathbf{x}} = \mathbf{1}[\mathbf{P}(\mathbf{e} - \boldsymbol{\mu}) > \boldsymbol{\tau}]$, and the public constants $(\mathbf{P}, \boldsymbol{\mu}, \boldsymbol{\tau})$ and the seed ship beside the model.

The map preserves similarity by the random-hyperplane LSH identity: for two unit vectors at angle θ a random hyperplane separates them with probability θ/π , so for the d -bit code

$$\frac{\mathbb{E}[\text{Hamming}(\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2)]}{d} = \frac{\theta}{\pi} = \frac{1}{\pi} \arccos(\langle \hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2 \rangle),$$

mapping genuine pairs (small θ) to small Hamming distance and impostor pairs (large θ) to large Hamming distance - exactly the quantity FLPSI thresholds. We use the *Super-Bit* construction rather than plain SimHash because orthonormalising the projection rows in blocks of $N = 16$ (the input dimension) leaves the estimator above unbiased while strictly lowering its variance at the acute angles that separate genuine from impostor pairs, giving a tighter Hamming separation at the *same* 128 bits and zero extra cost; at block size 1 it degenerates to plain SimHash, which it therefore strictly subsumes. A fourth, *optional* stage may be inserted before projection - rescaling the dimensions by $\text{diag}(\sqrt{|\mathbf{w}_1|})$, the per-dimension weights of the head’s final layer, so that Hamming distance tracks the learned accept metric rather than raw cosine - but we find it helps only an under-trained embedding and *hurts* a well-trained one (§7.2), and our recommended configuration omits it.//

Calibrated on held-out identities, the code shows the wide separation the design needs: genuine pairs concentrate at Hamming distance $\approx 8/128$ and impostor pairs at $\approx 64/128$, with output bits balanced at 50.2% and a Fisher separation $d' = (\mu_{\text{imp}} - \mu_{\text{gen}}) / \sqrt{(\sigma_{\text{gen}}^2 + \sigma_{\text{imp}}^2)/2} = 3.78$; a plain Hamming threshold $d_k \approx 20$ sits between the two modes. FLPSI does not test Hamming $\leq d_k$ directly, however: it accepts when at least t of T random sub-samples collide, a smoother implicit threshold on total Hamming distance. With H the Hamming distance and w the number of positions in one sub-sampling mask, a single sub-sample collides with probability $q(H) = \binom{d-H}{w} / \binom{d}{w}$ and the pair is accepted with probability $\Pr[\text{Binomial}(T, q(H)) \geq t]$. We

therefore calibrate (w, t, T) rather than a raw d_k : with $T = 64$ and t kept small (the Shamir secret-sharing cost grows with t), a mask weight $w = 14$ places the operating point in the genuine/impostor gap above. The full ROC sweep over (w, t) and the multi-finger fusion that turn this single-finger code into a deployable operating point lie beyond the quantiser itself - they are operating-point and system-level questions, treated with the evaluation in §7.//

It is tempting to argue that publishing $\mathbf{P}$ is harmless because recovering $\mathbf{e}$ from $\tilde{\mathbf{x}}$ would mean inverting an underdetermined system. *This reasoning is backwards, and we flag it explicitly.* The map applies 128 sign measurements to a 16-dimensional vector - an *over*-determined, roughly $8\times$ oversampled system - and one-bit compressed sensing recovers the *direction* of $\mathbf{e}$ from public $\mathbf{P}$ and $\tilde{\mathbf{x}}$ up to an angular error that shrinks with the oversampling ratio (the magnitude is lost but irrelevant after L2-normalisation). The sign map is therefore *not* a privacy mechanism and must not be relied on as one. Biometric privacy rests instead entirely on the FLPSI layer: the server never receives $\tilde{\mathbf{x}}$ in the clear, only PRF-masked sub-samples through the masked-OPRF and TLPSI sub-protocols, so it cannot assemble the code that the inversion would require. C2’s responsibility is accuracy and format; secrecy is discharged by the protocol of §4.4 and analysed in §5.

4.3 Registration Protocol

The registration phase proceeds as follows:

1. The user captures a reference fingerprint image on their device.
2. The client runs the CNN feature extractor to produce the floating-point embedding, followed by the quantisation bridge to produce the 128-bit binary template $\tilde{\mathbf{y}}_u$.
3. The client sends $\tilde{\mathbf{y}}_u$ along with a unique identifier ℓ_u to the authentication server over a secure channel (e.g., TLS).
4. The server stores the pair $(\tilde{\mathbf{y}}_u, \ell_u)$ in plaintext in its database.

Because the 128-bit binary template is essentially a randomized projection of the embedding, the raw fingerprint image and the dense floating-point vector never leave the user’s device. Compared to Blind-Touch, which must store heavy CKKS ciphertexts (≈ 855 KB packed per 512 users), our approach stores just 16 bytes per user in plaintext, reducing storage overhead by orders of magnitude.

4.4 Authentication Protocol

The authentication framework proposed in this work relies on the Fuzzy Labelled Private Set Intersection (FLPSI) protocol to perform privacy-preserving approximate matching between biometric templates. Unlike conventional Private Set Intersection protocols that require exact equality between two inputs, FLPSI allows two binary feature vectors to be considered identical even when they differ in a limited number of bit positions. This property makes FLPSI particularly suitable for biometric authentication, where two measurements of the same fingerprint rarely produce identical binary representations due to sensing noise, finger placement, and environmental variations.

Rather than directly comparing the complete feature vectors, FLPSI transforms the approximate matching problem into a Threshold Labelled Private Set Intersection (TLPSI) problem. The transformation enables approximate matching to be securely evaluated using efficient cryptographic primitives while revealing only the associated label of a matching database entry. This section presents the complete FLPSI construction, beginning with the fuzzy matching problem, followed by the TLPSI formulation, and finally the cryptographic protocol used to securely realize the functionality.

4.4.1 Fuzzy Matching Problem

Consider two parties, namely a client $\mathcal{C}$ and a server $\mathcal{S}$.

The client possesses a binary feature vector

$$\mathbf{x} \in \{0, 1\}^d,$$

where d denotes the feature dimension.

The server maintains a database

$$\mathcal{D} = \{(\mathbf{y}_1, \ell_1), (\mathbf{y}_2, \ell_2), \dots, (\mathbf{y}_N, \ell_N)\},$$

where

$$\mathbf{y}_i \in \{0, 1\}^d$$

represents the enrolled binary template of the i^{th} user and

$$\ell_i$$

denotes the corresponding label.

The objective of the authentication protocol is to determine whether there exists a template satisfying

$$d_H(\mathbf{x}, \mathbf{y}_i) \leq d_k,$$

where

$$d_H(\cdot, \cdot)$$

denotes the Hamming distance and

$$d_k$$

is the maximum acceptable distance.

If such a template exists, the protocol returns

$$\ell_i.$$

Otherwise,

$$\perp$$

is returned.

Traditional Private Set Intersection protocols assume that two inputs are considered identical only when every corresponding element is equal. Formally,

$$\mathbf{x} = \mathbf{y}_i.$$

This assumption is generally unsuitable for biometric authentication because repeated measurements of the same fingerprint often produce slightly different feature vectors. As a result, directly applying exact PSI would reject many genuine authentication attempts.

Instead, the authentication problem must tolerate a limited number of bit differences while still preserving the privacy of both participants. FLPSI achieves this objective by reducing the fuzzy matching problem to a Threshold Labelled Private Set Intersection problem.

4.4.2 Shamir Threshold Secret Sharing

Threshold Labelled Private Set Intersection (TLPSI) relies on Shamir’s Threshold Secret Sharing scheme to protect the server’s label while ensuring that it can only be recovered when a sufficient number of matching elements exist between the client and the server. This subsection briefly introduces the underlying secret sharing mechanism.

Suppose a dealer wishes to distribute a secret

$$s \in \mathbb{F}_q,$$

among multiple participants such that any collection of at least t participants can recover the secret, whereas any group containing fewer than t participants obtains no information about it. This property is referred to as a (t, n) *threshold secret sharing scheme*.

To construct the shares, the dealer first generates a random polynomial

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1},$$

of degree $t - 1$, where

$$a_0 = s,$$

and the remaining coefficients

$$a_1, a_2, \dots, a_{t-1}$$

are sampled uniformly at random from the finite field $\mathbb{F}_q$.

Each participant is then assigned a distinct non-zero identifier x_i , and receives the corresponding share

$$(x_i, f(x_i)).$$

Since the polynomial has degree $t - 1$, any collection of at least t distinct shares uniquely determines the polynomial through polynomial interpolation. Consequently, the secret is recovered by evaluating the reconstructed polynomial at

$$f(0) = s.$$

Conversely, any collection containing fewer than t shares is insufficient to reconstruct the polynomial because infinitely many degree- $(t - 1)$ polynomials can pass through fewer than t points. Therefore, no information about the secret can be inferred from an insufficient number of shares.

In Bui and Cong’s TLPSI protocol [3], the server embeds a protected version of the user’s label as the constant coefficient of the secret-sharing polynomial. Every common element between the client’s and the server’s sets reveals one valid evaluation of this polynomial. Only when the number of common elements reaches the required threshold can the client reconstruct the polynomial and recover the protected label. This property enables TLPSI to reveal labels only for sufficiently similar biometric templates while preserving the privacy of all non-matching entries.

4.4.3 Threshold Labelled PSI Construction

This subsection recalls, for self-containedness, the cryptographic construction of Bui and Cong [3] that realizes the Threshold Labelled Private Set Intersection (TLPSI) functionality introduced in the previous section. *The construction, its correctness argument and its security proof are entirely theirs; we restate them here only because our parameter analysis in Section 8 depends on*

their internals. Their protocol combines Shamir's Threshold Secret Sharing with their Vector Ring Oblivious Linear Evaluation (Vector Ring-OLE) primitive to securely determine whether the intersection between the client and server sets satisfies the required threshold.

Assume that the client holds a private set

$$X = \{x_1, x_2, \dots, x_d\},$$

while the server stores

$$Y = \{y_1, y_2, \dots, y_d\},$$

together with an associated label

$$\ell.$$

The objective of the protocol is to reveal the label only when

$$|X \cap Y| \geq t,$$

without revealing any additional information regarding either participant's private input.

Polynomial Construction The client first represents its input set by constructing a polynomial whose roots correspond to the elements contained in its set.

For the input

$$X = \{x_1, x_2, \dots, x_d\},$$

the client computes

$$u_0(x) = \prod_{i=1}^d (x - x_i).$$

Consequently,

$$u_0(x_i) = 0, \quad \forall x_i \in X.$$

The polynomial

$$u_0(x)$$

therefore uniquely represents the client's input set through its roots.

The server independently samples a uniformly random polynomial

$$u_1(x)$$

of degree

$$d.$$

Next, the server derives a masked representation of the associated label. Let

$$\ell' = \text{PRF}_k(\ell),$$

where

$$k$$

is the secret key of the pseudorandom function.

Instead of embedding the label directly, the protocol first constructs a Shamir secret sharing polynomial

$$f(x)$$

of degree

$$t - 1$$

such that

$$f(0) = \ell'.$$

The constant coefficient of

$$f(x)$$

therefore corresponds to the protected label, while the remaining coefficients are selected uniformly at random.

The server then generates a polynomial

$$v_1(x)$$

of degree

$$2d$$

that satisfies

$$v_1(y_i) = f(y_i), \quad \forall y_i \in Y.$$

Since the polynomial is constrained only at the points contained in the server's set, the remaining coefficients are selected randomly. Consequently,

$$v_1(x) = f(x) + \left(\prod_{i=1}^d (x - y_i) \right) R(x),$$

where

$$R(x)$$

is a uniformly random polynomial of degree

$$d.$$

This representation guarantees that

$$v_1(y_i) = f(y_i)$$

for every

$$y_i \in Y,$$

while preserving the secrecy of both the polynomial and the embedded label. Once the client polynomial

$$u_0(x)$$

and the server polynomials

$$u_1(x) \quad \text{and} \quad v_1(x)$$

have been constructed, both parties invoke the Vector Ring-OLE functionality introduced in the previous section.

The functionality securely computes

$$v_0(x) = u_0(x) u_1(x) + v_1(x),$$

without revealing either party's polynomial.

The client receives only

$$v_0(x),$$

whereas the server learns only the randomly generated polynomial

$$u_1(x).$$

Suppose

$$x_i \in X \cap Y.$$

Since

$$u_0(x_i) = 0,$$

the evaluated polynomial satisfies

$$v_0(x_i) = v_1(x_i) = f(x_i).$$

Therefore, every common element between the client and server sets reveals one valid evaluation of the Shamir secret sharing polynomial.

Conversely, if

$$x_i \notin Y,$$

the polynomial

$$v_1(x_i)$$

contains the additional random masking term

$$\left(\prod_{j=1}^d (x_i - y_j) \right) R(x_i),$$

which prevents the client from recovering a valid evaluation of

$$f(x).$$

If the number of common elements satisfies

$$|X \cap Y| \geq t,$$

the client obtains at least

$$t$$

correct evaluations of

$$f(x),$$

which are sufficient to reconstruct the secret-sharing polynomial using Shamir interpolation. The recovered secret corresponds to the masked label

$$\ell',$$

from which the original label can subsequently be recovered. The complete label recovery procedure is presented in the following subsection.

Label Recovery At the end of the Vector Ring-OLE protocol, the client obtains the polynomial

$$v_0(x) = u_0(x)u_1(x) + v_1(x),$$

while the server retains the randomly generated polynomial

$$u_1(x).$$

Recall that for every common element

$$x_i \in X \cap Y,$$

the client obtains

$$v_0(x_i) = f(x_i),$$

since

$$u_0(x_i) = 0.$$

Therefore, every common element contributes one valid evaluation point of the secret-sharing polynomial

$$f(x).$$

Let

$$\mathcal{P} = \{(x_i, v_0(x_i)) : x_i \in X \cap Y\}$$

denote the set of recovered evaluation points.

If the number of recovered evaluation points satisfies

$$|\mathcal{P}| \geq t,$$

the client possesses at least t distinct points on the polynomial

$$f(x),$$

whose degree is

$$t - 1.$$

According to the properties of Shamir's Threshold Secret Sharing introduced in Section 2.2, a polynomial of degree

$$t - 1$$

is uniquely determined by any collection of

$$t$$

distinct evaluation points. Consequently, the client reconstructs

$$f(x)$$

using polynomial interpolation.

Once the polynomial has been reconstructed, the client evaluates

$$f(0),$$

which corresponds to the protected label

$$\ell' = \text{PRF}_k(\ell).$$

Since the value stored in the polynomial is the output of a pseudorandom function rather than the original label, revealing

$$f(0)$$

does not disclose the server's label directly.

To enable the client to recover the original label while preventing exhaustive search attacks, the server additionally computes

$$h_\ell = H_1(\ell')$$

and

$$\Delta_\ell = H_2(\ell') \oplus \ell,$$

where

$$H_1$$

and

$$H_2$$

are collision-resistant hash functions, and

$$\oplus$$

denotes the bitwise exclusive-or operation.

After reconstructing

$$\ell',$$

the client first verifies its correctness by checking whether

$$H_1(\ell') = h_\ell.$$

If the verification succeeds, the original label is recovered as

$$\ell = \Delta_\ell \oplus H_2(\ell').$$

Otherwise, the reconstruction is considered unsuccessful and the protocol outputs

$\perp$.

The verification step prevents the client from accepting an incorrectly reconstructed polynomial, which may arise when fewer than t valid evaluation points are available. Furthermore, because the reconstructed value is first mapped through a pseudorandom function before being combined with the label, the protocol prevents the client from performing dictionary attacks against labels belonging to a small domain. Consequently, the server reveals the label only when the threshold condition is satisfied, while preserving the privacy of all non-matching database entries.

The Threshold Labelled Private Set Intersection protocol described above assumes that the client and server already possess sets whose common elements correspond to matching features. However, biometric templates are represented as binary feature vectors rather than ordinary sets. The following section introduces the masked Oblivious Pseudorandom Function (mOPRF), which transforms the sampled binary feature vectors into private set representations while preventing either party from learning the intermediate values. This transformation enables the TLPSI protocol to be applied directly to privacy-preserving biometric matching.

Algorithm 1 Threshold Labelled Private Set Intersection (TLPSI)

Input:

Client:

$$X = \{x_1, x_2, \dots, x_n\}$$

Server:

$$Y = \{y_1, y_2, \dots, y_m\},$$

with an associated label ℓ for each server element and a matching threshold t .

Output:

The client recovers the server label only if

$$|X \cap Y| \geq t.$$

- 1: Server encodes each label using Shamir's Threshold Secret Sharing.
- 2: Server constructs the corresponding secret-sharing polynomial.
- 3: Client constructs a polynomial representing its input set.
- 4: Server constructs polynomials corresponding to its labelled set.
- 5: Client and server execute the Vector Ring-OLE protocol.
- 6: Client receives polynomial evaluations corresponding to the intersection elements.
- 7: If fewer than t valid evaluations are obtained,
 terminate and output $\perp$.
- 8: Reconstruct the secret-sharing polynomial using polynomial interpolation.
- 9: Recover the label by evaluating the reconstructed polynomial at

$$f(0).$$

- 10: Output the recovered label.
-

Algorithm 1 summarizes the complete execution flow of the Threshold Labelled Private Set Intersection protocol. The server first protects each label using Shamir's Threshold Secret Sharing, ensuring that the label can only be reconstructed from at least t valid evaluations. Subsequently, both parties execute the Vector Ring-OLE protocol to securely evaluate the required polynomials without revealing their underlying sets. Finally, if the number of common elements reaches the predefined threshold, the client reconstructs the secret-sharing polynomial

through polynomial interpolation and recovers the associated label; otherwise, no information about the label is revealed.

4.4.4 Pseudorandom Function

The Fuzzy Labelled Private Set Intersection (FLPSI) protocol employs a Pseudorandom Function (PRF) to securely encode biometric sub-samples and to protect the labels associated with enrolled biometric templates. Unlike a conventional hash function, a PRF requires a secret key, ensuring that only parties possessing the key can correctly evaluate the function.

Formally, let

$$F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$$

be a keyed function, where $\mathcal{K}$ denotes the key space, $\mathcal{X}$ the input domain, and $\mathcal{Y}$ the output range. Given a secret key

$$k \in \mathcal{K},$$

the function

$$F_k(x)$$

maps every input

$$x \in \mathcal{X}$$

to a deterministic output in $\mathcal{Y}$.

A secure pseudorandom function satisfies two important properties.

- **Determinism:** Evaluating the function using the same secret key and the same input always produces the same output. Consequently,

$$F_k(x) = F_k(x),$$

allowing different parties possessing the same key to independently compute identical outputs.

- **Pseudorandomness:** Without knowledge of the secret key, the output of the function is computationally indistinguishable from a uniformly random value. Therefore, an adversary observing $F_k(x)$ cannot efficiently recover either the secret key or the original input.

These properties make PRFs particularly suitable for privacy-preserving set intersection protocols. Equal inputs evaluated under the same secret key produce identical outputs, thereby preserving equality, while different inputs appear as random and unrelated values to any party without knowledge of the key.

In Bui and Cong’s authentication framework [3], the PRF serves two distinct purposes. First, it transforms sampled biometric features into pseudorandom values before they participate in the Threshold Labelled Private Set Intersection protocol, preventing either participant from learning the underlying biometric information. Second, the server applies the PRF to each user’s label before embedding it into the Shamir secret-sharing polynomial, ensuring that the reconstructed value does not directly reveal the original label. The masked Oblivious Pseudorandom Function described in the following subsection builds upon this primitive to securely evaluate the PRF without exposing either the secret key or the client’s biometric data.

4.4.5 Masked Oblivious Pseudorandom Function

The Threshold Labelled Private Set Intersection protocol described in the previous subsection assumes that both parties already possess private sets whose common elements correspond to matching features. In the biometric authentication setting, however, the client and the server initially hold binary feature vectors rather than sets. A direct conversion from binary vectors to sets by applying publicly known sub-sampling masks would expose portions of the underlying biometric template whenever a match occurs.

To eliminate this leakage, the FLPSI protocol introduces a *Masked Oblivious Pseudorandom Function* (mOPRF). The mOPRF simultaneously performs sub-sampling and pseudorandom encoding of the sampled values, ensuring that neither party learns the intermediate sub-samples in plaintext while still producing consistent outputs for identical masked inputs.

Let

$$\mathbf{x} \in \{0, 1\}^d$$

denote the client's binary feature vector and

$$\mathbf{y} \in \{0, 1\}^d$$

denote one server template.

Suppose the parties publicly agree on a collection of sampling masks

$$\mathcal{M} = \{m_1, m_2, \dots, m_T\},$$

where every mask

$$m_j \in \{0, 1\}^d$$

contains exactly

$$d_s$$

non-zero positions.

A straightforward approach would construct the sampled subsets

$$\mathbf{x} \wedge m_j, \quad \mathbf{y} \wedge m_j,$$

for every

$$j \in [T],$$

where

$$\wedge$$

denotes the bitwise AND operation.

Although this approach successfully converts the feature vectors into collections of subsets, the sampled values themselves become visible whenever a successful intersection occurs. Consequently, the client may learn partial information regarding the server's enrolled biometric template, violating the desired privacy guarantees of the authentication protocol.

To prevent this leakage, every sampled subset is first processed by a pseudorandom function before participating in the Threshold Labelled PSI protocol.

Definition 2 (Masked Oblivious Pseudorandom Function). *Let*

$$F : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{D}$$

denote a secure pseudorandom function.

The receiver inputs

$$\mathbf{x} \in \{0, 1\}^d,$$

while the sender inputs

$$k \in \mathcal{K},$$

together with a collection of sampling masks

$$\mathcal{M} = \{m_1, m_2, \dots, m_T\}.$$

The functionality returns to the receiver

$$X' = \{F_k(\mathbf{x} \wedge m_1), F_k(\mathbf{x} \wedge m_2), \dots, F_k(\mathbf{x} \wedge m_T)\},$$

without revealing either the secret key

$$k$$

or the masked inputs.

Since the sender possesses both the secret key

$$k$$

and the collection of sampling masks, it locally computes the corresponding masked outputs for every enrolled template

$$\mathbf{y}_i \in \mathcal{D}.$$

For the

$$i^{\text{th}}$$

database entry, the sender constructs

$$Y'_i = \{F_k(\mathbf{y}_i \wedge m_1), F_k(\mathbf{y}_i \wedge m_2), \dots, F_k(\mathbf{y}_i \wedge m_T)\}.$$

Thus, every enrolled biometric template is transformed into a set of pseudorandom values that can subsequently participate in the Threshold Labelled PSI protocol.

An important property of the pseudorandom function is determinism. Whenever the client and the server possess identical masked sub-samples,

$$\mathbf{x} \wedge m_j = \mathbf{y}_i \wedge m_j,$$

both parties obtain the same pseudorandom output

$$F_k(\mathbf{x} \wedge m_j) = F_k(\mathbf{y}_i \wedge m_j).$$

Conversely, distinct masked inputs produce computationally indistinguishable pseudorandom values with overwhelming probability. Consequently, the equality of two masked sub-samples is preserved without revealing their underlying binary representation.

After executing the masked OPRF protocol, the client obtains the transformed set

$$X',$$

while the server computes the corresponding transformed sets

$$\{Y'_1, Y'_2, \dots, Y'_N\}.$$

These pseudorandom sets are subsequently used as the inputs to the Threshold Labelled PSI protocol described in Section 3.3. Since identical sub-samples remain equal after the pseudorandom transformation, the threshold intersection property is preserved while preventing either party from learning the sampled biometric bits in plaintext.

4.4.6 Fuzzy Labelled Private Set Intersection Protocol

The complete FLPSI protocol combines the masked Oblivious Pseudorandom Function introduced in the previous subsection with the Threshold Labelled Private Set Intersection protocol described in Section 3.3. The masked OPRF converts binary feature vectors into pseudorandom set representations, while the TLPSI protocol securely determines whether these transformed sets satisfy the required threshold condition.

Let the client possess a binary feature vector

$$\mathbf{x} \in \{0, 1\}^d,$$

and let the server maintain the labelled database

$$\mathcal{D} = \{(\mathbf{y}_1, \ell_1), (\mathbf{y}_2, \ell_2), \dots, (\mathbf{y}_N, \ell_N)\},$$

where

$$\mathbf{y}_i \in \{0, 1\}^d$$

is the enrolled binary feature vector of the i^{th} user and

$$\ell_i$$

is the corresponding label.

The protocol proceeds in two stages. During the first stage, the parties execute the masked OPRF protocol to obtain pseudorandom representations of the sampled feature vectors. During the second stage, these pseudorandom sets are supplied to the Threshold Labelled PSI protocol to determine whether the number of matching sub-samples exceeds the predefined threshold.

Initialization Before protocol execution, the server samples a uniformly random secret key

$$k \leftarrow \mathcal{K},$$

for the pseudorandom function

$$F_k(\cdot).$$

The server also generates a family of sampling masks

$$\mathcal{M} = \{m_1, m_2, \dots, m_T\},$$

where every mask

$$m_j \in \{0, 1\}^d$$

contains exactly

$$d_s$$

non-zero positions.

The parameters

$$(T, d_s, t)$$

are selected during system initialization to achieve the desired balance between the false acceptance rate and the false rejection rate. These masks remain fixed throughout protocol execution and are reused for all authentication sessions.

Pseudorandom Sub-sampling The client and the server jointly execute the masked OPRF protocol using the feature vector

$$\mathbf{x}$$

and the collection of sampling masks

$$\mathcal{M}.$$

At the completion of the protocol, the client receives

$$X' = \{F_k(\mathbf{x} \wedge m_1), F_k(\mathbf{x} \wedge m_2), \dots, F_k(\mathbf{x} \wedge m_T)\},$$

without learning either the secret key

$$k$$

or the sampling masks used internally by the protocol.

Meanwhile, the server locally evaluates the pseudorandom function for every enrolled template to obtain

$$Y'_i = \{F_k(\mathbf{y}_i \wedge m_1), F_k(\mathbf{y}_i \wedge m_2), \dots, F_k(\mathbf{y}_i \wedge m_T)\},$$

for every

$$i \in [N].$$

The resulting pseudorandom values replace the original sampled feature vectors and serve as the inputs to the Threshold Labelled PSI protocol.

A practical issue may arise when two different sampling masks produce identical pseudorandom outputs. Such duplicate values become more likely when the sampling size is small or when a large number of masks are employed. Duplicate elements may incorrectly increase the intersection size and therefore affect the correctness of the threshold comparison.

To eliminate this ambiguity, every pseudorandom output is combined with the index of the corresponding sampling mask. Specifically, the client transforms its input into

$$X'' = \{X'_1 \oplus 1, X'_2 \oplus 2, \dots, X'_T \oplus T\},$$

while the server performs the identical transformation

$$Y''_i = \{Y'_{i,1} \oplus 1, Y'_{i,2} \oplus 2, \dots, Y'_{i,T} \oplus T\}.$$

Since each sub-sample is now uniquely identified by its sampling position, duplicate values generated from different masks can no longer collide.

Threshold Matching For every enrolled biometric template, the server and the client execute the Threshold Labelled Private Set Intersection protocol using

$$X''$$

and

$$Y_i''.$$

The TLPSI protocol determines the cardinality of

$$X'' \cap Y_i'',$$

without revealing any additional information regarding either participant's private input. If

$$|X'' \cap Y_i''| \geq t,$$

the client successfully reconstructs the protected label

$$\ell_i$$

using the label recovery procedure described in Section 3.3.

Otherwise,

$$\perp$$

is returned, indicating that the corresponding enrolled template does not satisfy the required similarity threshold.

The correctness of the protocol follows from two observations.

First, if two biometric feature vectors differ in at most

$$d_k$$

bit positions, the probability that at least

$$t$$

sampled subsets remain identical is overwhelmingly high for the selected protocol parameters.

Second, the masked OPRF preserves equality between identical masked sub-samples while hiding their underlying binary representation. Consequently, identical biometric regions produce identical pseudorandom values, enabling the Threshold Labelled PSI protocol to identify sufficiently similar feature vectors without revealing any biometric information.

The FLPSI protocol described above specifies the logical functionality required for privacy-preserving biometric authentication. The computational efficiency of the protocol is achieved through the Vector Ring Oblivious Linear Evaluation primitive introduced in Section 2, which realizes the underlying Threshold Labelled PSI functionality using a single Vector Ring-OLE instance. This optimization significantly reduces both the communication overhead and the on-line computation compared with previous constructions while preserving the security guarantees of the protocol.

Algorithm 2 Fuzzy Labelled Private Set Intersection (FLPSI)

Input:

Client fingerprint feature vector

$$B_c \in \{0, 1\}^N$$

Server database

$$\mathcal{D} = \{(B_i, \ell_i)\}_{i=1}^M$$

Matching threshold

$$t$$

Output:

Authentication result and the corresponding label.

- 1: Extract the binary fingerprint feature vector from the client fingerprint.
- 2: Generate the predefined binary masks.
- 3: Apply each mask to obtain multiple fingerprint sub-samples.
- 4: Execute the masked Oblivious Pseudorandom Function (mOPRF) on every sub-sample.
- 5: Construct the transformed client set

$$X.$$

- 6: **for** each enrolled fingerprint (B_i, ℓ_i) in the server database **do**
- 7: Generate the corresponding masked sub-samples.
- 8: Evaluate the mOPRF on every server sub-sample.
- 9: Construct the transformed server set

$$Y_i.$$

- 10: Execute the Threshold Labelled Private Set Intersection (TLPSI) protocol between X and Y_i .
 - 11: **if** the reconstructed label is valid **then**
 - 12: Authenticate the client.
 - 13: Output the recovered label.
 - 14: **return**
 - 15: **end if**
 - 16: **end for**
 - 17: Reject the authentication request.
-

Algorithm 2 summarizes the complete execution flow of the proposed fingerprint authentication framework. The client first extracts its fingerprint feature vector, generates masked sub-samples, and transforms them using the masked Oblivious Pseudorandom Function. Similarly, the server preprocesses each enrolled fingerprint template using the same masking strategy and evaluates the corresponding mOPRF values. The client and server subsequently execute the Threshold Labelled Private Set Intersection protocol, which reveals a protected label only when the number of matching transformed features satisfies the predefined threshold. Successful reconstruction of the label indicates a valid authentication, whereas failure to reconstruct any label results in rejection of the authentication request.

5 Security Analysis

We work in the semi-honest model with static corruptions. Throughout, λ is the computational security parameter, κ the statistical one, $\mathbb{F}$ a field with $|\mathbb{F}| \geq 2^{\kappa + \log N}$, and negl an unspecified negligible function. Section 5.5 then steps outside that model, because the most serious weakness we identify is not a protocol weakness at all.

5.1 The functionality

Prior work in this line proves security for *one* client set against *one* labelled server set. NeuralPSI runs N record instances sharing a receiver polynomial and a PRF key, so the composite object needs to be named before anything can be proved about it.

Functionality $\mathcal{F}_{\text{NeuralPSI}}^{d,T,w,\theta}$

Parties. A client C and a server S .

Parameters. Code length d , sub-sample count T , sub-sample width w , acceptance threshold θ , and a public mask family $M = \{m_j\}_{j \in [T]}$, $m_j \in \{0, 1\}^d$, $\|m_j\|_1 = w$.

Inputs. C inputs $\tilde{\mathbf{x}} \in \{0, 1\}^d$. S inputs a database $D = \{(\tilde{\mathbf{y}}_i, \ell_i)\}_{i \in [N]}$ with $\tilde{\mathbf{y}}_i \in \{0, 1\}^d$ and labels $\ell_i \in \mathbb{F}$.

Computation. For each $i \in [N]$ let

$$S_i = \{j \in [T] : \tilde{\mathbf{x}} \wedge m_j = \tilde{\mathbf{y}}_i \wedge m_j\}, \quad \text{acc}(i) = [|S_i| \geq \theta].$$

Outputs. C receives $\text{out} = \{(i, \ell_i) : \text{acc}(i)\}$; S receives $\perp$.

Figure 1: The composite functionality. Note that the output is a *set*: the scan runs over all N records and more than one may satisfy the predicate. This differs from the single-match phrasing used in earlier drafts of this work, and the difference is observable — $|\text{out}| > 1$ occurs at a rate we measure in Section 8.

Two features of Figure 1 deserve emphasis. First, acceptance is a deterministic function of $\tilde{\mathbf{x}}$, $\tilde{\mathbf{y}}_i$ and the public mask family, so it depends on the pair only through which positions differ; because the masks are drawn uniformly it depends in distribution only on the Hamming weight $H = \|\tilde{\mathbf{x}} \oplus \tilde{\mathbf{y}}_i\|_1$. Every error rate in Section 8 follows from that observation. Second, the functionality says nothing about *authentication*: no party other than C learns anything, so a relying party cannot act on the result. We return to this in Section 5.6.

The masks are public. We fix this now because the leakage analysis depends on it. Taking M public is the conservative reading: it gives the adversary strictly more than a server-private mask family would, and our claims hold under it. Making M server-private would shrink $\mathcal{L}_C$ below, but the protocol as constructed delivers the collision pattern to the client, so a private M would require a structural change rather than a parameter choice.

5.2 The leakage function

$\mathcal{F}_{\text{NeuralPSI}}$ is what we would like to realise. What the protocol realises is $\mathcal{F}_{\text{NeuralPSI}}$ *augmented with leakage*:

$$\mathcal{L}_C(\tilde{\mathbf{x}}, D) = \left(N, \{ (i, S_i) : |S_i| \geq \theta \} \right), \tag{1}$$

$$\mathcal{L}_S(\tilde{\mathbf{x}}, D) = \perp. \tag{2}$$

Equation (1) is strictly more than “the identity of the matched record”. The client learns *which* sub-samples collided, and since M is public, each $j \in S_i$ pins w known positions of $\tilde{\mathbf{y}}_i$ to known values. The client also learns N , the number of records tested. We quantify the consequences in Section 5.7; the short version is that $\mathcal{L}_C$ is enough to reconstruct a matched record outright.

5.3 Security theorem

Theorem 1 (Semi-honest security). *Let F be a PRF, and instantiate the sub-protocols with a masked OPRF Π_{mOPRF} and a vector ring-OLE Π_{VROLE} , each semi-honest secure. Then the NeuralPSI protocol of Section 4 securely realises $\mathcal{F}_{\text{NeuralPSI}}^{d,T,w,\theta}$ with leakage $(\mathcal{L}_C, \mathcal{L}_S)$ against a semi-honest adversary statically corrupting one party. Concretely, for every such adversary $\mathcal{A}$ there is a simulator whose output is distinguishable from $\mathcal{A}$ ’s view with advantage at most*

$$\varepsilon \leq \varepsilon_{\text{mOPRF}} + \varepsilon_{\text{VROLE}} + \varepsilon_{\text{LPN}} + \underbrace{\varepsilon_F(q + T \cdot 2^w)}_{\text{PRF, extended domain}} + \underbrace{\frac{N(2T+1)}{|\mathbb{F}|}}_{\text{statistical}}, \quad (3)$$

where q is the adversary’s own query count.

Two terms in (3) are easy to omit and both matter. The PRF term is charged for $T \cdot 2^w$ extra oracle queries, not q alone: each mask restricts the PRF input to a domain of size $2^w = 2^{14}$, so the full input table has only $T \cdot 2^w = 2^{20}$ entries and the reduction must budget for enumerating it. The statistical term is the price of the polynomial masking step (hybrid H_4 below) and is the reason $|\mathbb{F}|$ must be chosen with N in view; we report the concrete field size in Section 7.

Both simulators are constructed in Appendix A. The corrupt-client simulator takes $\tilde{\mathbf{x}}$ and $(\text{out}, \mathcal{L}_C)$ and produces the mOPRF outputs, the sender-side silent-VOLE transcript and the polynomials $\{\hat{z}_i\}$, sampling uniformly at every point not pinned by $\mathcal{L}_C$. The corrupt-server simulator has only the transcript to produce, since the server outputs $\perp$; its one input-dependent message is $\Delta U = \Delta u(X)$. Worth stating precisely, because the natural argument is slightly wrong: ΔU is *not* uniform on $\mathbb{F}^{2T+1}$ but lies in the $(T+1)$ -dimensional image of the degree- $\leq T$ evaluation map, so the simulator samples a uniform polynomial of degree $\leq T$ and outputs its evaluation vector. Since u'_0 is uniform, $u_0 - u'_0$ is uniform *in that subspace* and the indistinguishability is statistical, not computational.

5.4 Hybrids

H_0 is the real execution. H_1 replaces Π_{mOPRF} by $\mathcal{F}_{\text{mOPRF}}^T$ (semi-honest mOPRF security); H_2 replaces F_k by a random function (PRF security, charged at $q + T \cdot 2^w$ queries as above); H_3 replaces Π_{VROLE} by $\mathcal{F}_{\text{VROLE}}$ (base-VOLE security and LPN).

H_4 is the load-bearing *statistical* step, and is the one most easily left unstated: for every non-colliding point x_j it replaces $v_1(x_j) = f(x_j) + (\prod_i (x_j - \tilde{\mathbf{y}}_i)) \cdot R(x_j)$ by a uniform element of $\mathbb{F}$. This holds because R is uniform of degree $\leq T$ and the evaluation map on any set of at most $T+1$ points is a bijection onto $\mathbb{F}^{T+1}$, so the masking terms are jointly uniform — *provided*

$$|\{j \in [T] : j \notin S_i\}| \leq \deg R + 1 = T + 1, \quad (4)$$

which holds unconditionally here since $|[T]| = T$, but is a design constraint rather than an accident. Over N records and $2T+1$ points the union bound gives the $N(2T+1)/|\mathbb{F}|$ term in (3). Finally H_5 replaces $\ell'_i = F_k(\ell_i)$ by uniform elements (immediate from H_2), and $H_6 = \text{Sim}_C$ is identical by inspection.

Why N instances do not compose for free. The step that a citation cannot discharge is the reuse across records. All N record instances share one receiver polynomial u_0 — that sharing is precisely what makes the vector ring-OLE batching asymmetric and cheap — and one PRF key k , across all records *and* all sessions. Hybrids H_2 and H_4 are therefore stated over the whole database rather than per record: H_2 replaces the single shared F_k once, which is why its cost appears once rather than N times, and H_4 is applied simultaneously at all $N(2T + 1)$ points with the union bound in (3). Constraint (4) must hold for every record at once; it does, but only because $\deg R \geq T$, which is a design requirement rather than an accident and is easy to violate when tuning T downward for communication.

The long-form hybrids, with the distinguisher reductions written out, are in Appendix A.

5.5 The stored template is a bearer credential

Theorem 1 covers the protocol and says nothing about data at rest, which is where this design is weakest — strictly weaker than the homomorphic baseline, whose stored feature vectors are encrypted under a key the server never holds.

The server stores $(\tilde{\mathbf{y}}_i, \ell_i)$ in the clear. An adversary who reads $\tilde{\mathbf{y}}_i$ and submits it as a query obtains $H = 0$, hence $|S_i| = T$ and acceptance with probability 1. *No inversion is required*: the stored code *is* a working credential, so a database breach is immediate and total impersonation of every enrolled user.

Inversion is an additional problem. The quantiser is over-determined — 128 hyperplanes cut $\mathbb{S}^{15}$ into $2 \sum_{k < 16} \binom{127}{k} \approx 2^{64.5}$ cells — so it is not one-way in any useful sense. Its intrinsic angular resolution is $\text{cells}^{-1/15} \approx 2.9^\circ$, and the standard one-bit compressed-sensing heuristic $k/m = 16/128$ gives 7.2° , against a measured genuine intra-class spread of $\pi \cdot 8.3/128 = 11.7^\circ$. Reconstruction is thus 1.6–4× *tighter* than genuine variation: the recovered direction re-quantises inside the accept region and transfers to any system using the same extractor.

Table 1: The stored template against ISO/IEC 24745 [1]. NeuralPSI as specified fails all three requirements.

Requirement	Status	Why
Irreversibility	fails	Twice: direct replay of the stored code, and one-bit compressed-sensing recovery of the embedding direction to $2.9\text{--}7.2^\circ$.
Unlinkability	fails	(P, μ, τ) and the projection seed are public and fixed, so one finger yields the same 128-bit code in every deployment and codes are directly Hamming-comparable across databases. There is no diversification parameter.
Renewability	fails	A second independent code cannot be issued from the same finger. This is the requirement the application most needs, since a compromised fingerprint cannot be reissued.

Mitigations, and what they actually buy. (i) *A client-side per-enrolment XOR mask*: the device holds $r_u \in \{0, 1\}^d$ and the server stores $\tilde{\mathbf{y}} \oplus r_u$. XOR preserves Hamming distance exactly, so **accuracy is unchanged** — every number in §7 carries over — and 16 bytes of client state buys unlinkability and renewability. It does *not* stop replay of a stolen record. (ii) *Envelope encryption at rest*, key in an HSM, decrypting to memory for the scan: unglamorous, orthogonal to the protocol, sound in the semi-honest model, and what we would deploy. (iii) We explicitly reject one tempting option — storing the mOPRF images $\{F_k(\tilde{\mathbf{y}}_i \wedge m_j)\}_j$ instead of the code. It is keyed and locality-preserving and the server computes those values anyway, but the per-mask domain is only $2^w = 2^{14}$, so anyone who can evaluate F_k inverts each image by exhaustive search; repairing it needs a per-record salt that destroys the equality-preservation the protocol depends on.

Applying (i) and (ii) yields unlinkability and renewability. **Full at-rest confidentiality against a compromised server remains out of scope**, and this posture is weaker than the encrypted-template baseline’s — a limitation we state rather than resolve.

5.6 Who is actually authenticated

As specified, no relying party learns the outcome, so no session can be granted: $\mathcal{F}_{\text{NeuralPSI}}$ is a private identification lookup, not an authentication protocol. The credential step of Section 4 closes the gap for one MAC, one nonce and 32 bytes, with the nonce and session id making σ non-replayable — without them one intercepted transcript is a permanent credential, since ℓ is long-lived.

That step has a consequence for how Section 8 should be read. A client cannot forge σ without ℓ_i , so system security reduces to recovering *some* ℓ_i , which is exactly the false-accept question. In a conventional matcher a false accept is one bad login; here it hands the attacker a **permanent per-user credential**. *The false-accept rate is a credential-compromise rate*, which is why we treat it as a security parameter rather than an accuracy metric.

5.7 Attacks

(a) Impersonation without search. The parameters were chosen against an attacker submitting *uniformly random* codes, for which the per-record accept probability is $q_{\text{rand}} = 2.954 \times 10^{-5}$. But the cheapest attack is not to guess: it is to present one’s own finger. Measured over 2.88×10^6 held-out impostor pairs, the per-record accept probability for a *real but wrong* finger is $q_{\text{bio}} = 4.578 \times 10^{-2}$ — 1,550 $\times$ larger. Expected queries to impersonate somebody, $1/(1 - (1 - q)^N)$:

N	512	5,000	10^5	10^6
random code (q_{rand})	67	7	1.05	1
zero-effort finger (q_{bio})	1	1	1	1

At every database size we measured, a single presentation of an unrelated finger impersonates *someone* with probability above 99%. Rate limiting does not mitigate an attack that succeeds on the first attempt.

(b) Template recovery from the collision pattern. On any accept, $\mathcal{L}_C$ gives the client S_i . With $\theta = 2$ the client can enumerate all $\binom{64}{2} = 2,016$ candidate pairs, interpolate, and test, so it learns S_i exactly and cheaply. Because the masks are public, each $j \in S_i$ pins $w = 14$ known positions of $\tilde{\mathbf{y}}_i$. At $H \approx 8$ a genuine query produces $T \cdot q(8) = 64 \times 0.38493 \approx 24.6$ collisions, covering $24.6 \times 14 \approx 345$ position-slots over 128 positions: **one successful authentication determines the target record**. Recovering a *chosen* user costs $1/q_{\text{bio}} \approx 22$ queries at the measured rate; recovering *a* user’s full template at $N \geq 10^5$ costs one.

(c) The mOPRF dictionary. Each mask restricts the PRF domain to $2^w = 2^{14}$, and the entire table is $T \cdot 2^w = 2^{20}$. A client learns one entry per mask per session, so a coupon-collector argument gives roughly 1.1×10^4 sessions for 50% coverage per mask — already sufficient, since 32 of 64 masks at 14 bits each over-determines a 128-bit code — and about 1.7×10^5 sessions for full coverage. “PRF-masked” thus provides at most $w = 14$ bits of preimage resistance per sub-sample against a querying client.

Required deployment controls. Consequently: (i) rate limiting per account *and* per relying party, with the explicit admission that it does not mitigate (a) at large N ; (ii) authenticated enrolment, so an attacker cannot enrol chosen codes to probe the space; (iii) per-epoch rotation of k and of the mask family, at the stated re-enrolment cost; (iv) sensor attestation as an explicit deployment assumption; and (v) the nonce binding of Section 5.6.

What we do not claim. Malicious security — cut-and-choose techniques exist for related structure-aware PSI settings [5], but they target geometric-ball rather than threshold predicates, so their overheads do not transfer and we do not estimate them. At-rest confidentiality against a compromised server. That the quantiser is one-way: it demonstrably is not, and no part of Theorem 1 relies on it. And that the measured false-accept rates are acceptable at the sizes we tested — §8 shows they are not, and why.

6 Implementation

The quantiser of §4.2 is implemented as a small, training-free NumPy module. A `fit` routine estimates the public constants $(\mu, \mathbf{P}, \tau)$ from a calibration split - drawing the Gaussian blocks from a fixed seed, orthonormalising each by QR, optionally pre-composing the whitening diagonal, and recording the per-bit medians - and serialises them beside the model; `transform` then reduces to a single 128×16 matrix multiply and a per-bit threshold, emitting a 128-bit (`uint8[128]`) code. The same configuration flags (centre / whiten / Super-Bit / balance) drive the ablation ladder of Table 2.

7 Experimental Evaluation

7.1 Datasets and Metrics

We train and evaluate our system on the **SOCOFing** (Sokoto Coventry Fingerprint) dataset, a public collection of 6,000 real fingerprint images belonging to 600 subjects. To test generalization under adverse conditions, the dataset includes synthetic alterations (Altered-Easy, Altered-Medium, Altered-Hard). Following the standard open-set evaluation protocol, we split the identities such that the test set contains identities entirely unseen during training.

The primary evaluation metric is the Equal Error Rate (EER), the threshold where the False Acceptance Rate (FAR) and False Rejection Rate (FRR) are equal. A lower EER indicates a better biometric representation. We also evaluate the system against the floating-point baseline ceiling established by systems like Blind-Touch.

7.2 Authentication Accuracy

The baseline Blind-Touch system achieves a $\sim 0.7\%$ EER on the SOKOTO/SOCOFing dataset using unconstrained floating-point matching. Our full 224×224 NeuralPSI feature extractor is comparable to this baseline (the intervals overlap) in plaintext (0.33% EER). When enforcing our strict privacy guarantees through the 128-bit Super-Bit quantization bridge, the accuracy drops marginally to 1.88% EER. This ≈ 1.55 percentage point increase in EER represents the direct information-theoretic cost of mapping 16 continuous dimensions into a rigid 128-bit Hamming space for FLPSI compatibility. Given that this trade-off completely removes the HE scaling bottleneck, the resulting sub-2% EER remains highly competitive for large-scale deployments.

Table 2 reports the equal-error rate (EER) of the quantiser against the floating-point ceiling - the score on which homomorphic encryption operates - on SOCOFing under an open-set

protocol (held-out identities; enrolment on the Real print, genuine probe on the Altered-Easy print; 1,200 genuine and 240,000 impostor pairs).

Table 2: Open-set EER on SOCOFing. The *float* row is the accuracy ceiling that homomorphic encryption matches on; every other row is a d -bit binary code fed to FLPSI. Lower is better.

Representation	Bits	96×96 / 6-epoch CNN	224×224 / 150-epoch CNN
Float features (<i>HE ceiling</i>)	-	2.50%	0.33%
sign(> 0)	16	10.86%	10.12%
ITQ	16	7.12%	5.55%
Super-Bit (recommended)	128	5.14%	1.88%
+ metric whitening	128	3.91%	2.98%

Three findings stand out. First, a true 128-bit Super-Bit code dominates both the naïve sign(> 0) baseline and a 16-bit ITQ code: expanding to 128 bits by orthonormalised random projection - rather than repeating or learning 16 bits - carries far more of the embedding’s discriminative information into Hamming space. Second, *the embedding, not the quantiser, is the bottleneck*: retraining the CNN at full 224×224 resolution drops the float ceiling from 2.50% to 0.33% and the Super-Bit code from 5.14% to 1.88%, leaving the binarisation cost (binary minus float) at only +1.55 percentage points of EER. Third, *metric whitening reverses sign with model quality*: it helps the weak 96-px model (5.14% $\rightarrow$ 3.91%) by reweighting noisy dimensions but hurts the well-trained one (1.88% $\rightarrow$ 2.98%), whose geometry is already well-conditioned, so our recommended configuration omits it. The residual ≈ 1.5 -point gap between the 128-bit code and the float ceiling is information-theoretic - 128 sign bits of a 16-dimensional vector carry at most ~ 16 bits of genuine information - and cannot be closed by a better quantiser, only by a wider embedding. These numbers are on SOCOFing Altered-Easy, a synthetic-alteration protocol; cross-sensor and cross-session evaluation is expected to be harder and remains a fair-evaluation caveat.

7.3 Communication Cost

We measured the query-time communication of the FLPSI matcher on the real protocol reference implementation, instrumented to count protocol bytes, at our operating point ($d = 128$, $T = 64$, weight = 14, $t = 2$), sweeping the database size N (Table 3). The garbled circuits and garbler labels are pre-shared once in setup, so the tracked figure is the online cost (oblivious-PRF + TLPSI), counting both directions on the channel.

Table 3: Online (per-authentication) communication vs. database size N , measured on the reference FLPSI implementation at the $d=128$, $T=64$, $w=14$, $t=2$ operating point.

N	100	1,000	5,000	10,000	50,000	extrapolated
Communication	0.40 MB	1.76 MB	7.77 MB	15.28 MB	75.40 MB	~ 150 MB @ 100 K

The cost is linear, $\text{comm} \approx 0.25 \text{ MB} + 1.54 \text{ KB} \cdot N$, i.e. $\sim 1.54 \text{ KB}$ per database record asymptotically. It extrapolates to $\sim 150 \text{ MB}$ at $N = 100 \text{ K}$ and $\sim 1.5 \text{ GB}$ at $N = 1 \text{ M}$, matching Bui–Cong’s reported 153 MB and 1533 MB: NeuralPSI inherits FLPSI’s communication profile essentially unchanged, because the quantiser contributes *nothing* to communication - it only fixes the 128-bit width the protocol already assumes. Replacing the HE layer removes both Blind-Touch’s 117 MB Galois key and its per-user ciphertext expansion. (These are single-host loopback measurements - protocol bytes, not wire latency.)

7.4 End-to-End Latency

Table 4 breaks one authentication into its phases. The client-side work - the CNN forward pass and the Super-Bit quantisation - totals under a millisecond on a GPU (RTX 4050), so the bridge this paper introduces is not the bottleneck; query-time latency is dominated by the server-side FLPSI online scan at ~ 0.16 ms per record. The FLPSI *offline* phase (VOLE-OT setup, garbling, TLPSI construction) is data-independent and runs before the query arrives, so it does not count against query-time latency.

Table 4: End-to-end latency by phase (one authentication). CNN and quantise are client-side; FLPSI is the real protocol binary on a single host.

Phase	$N = 1,000$	$N = 5,000$	scaling
CNN forward (client, GPU)	0.80 ms	0.80 ms	$O(1)$ in N
Quantise (client)	<0.01 ms	<0.01 ms	$O(1)$ in N
FLPSI online (query-time)	187 ms	823 ms	~ 0.16 ms/record
Total query-time	~ 188 ms	~ 824 ms	-
FLPSI offline (preprocessing)	553 ms	1,946 ms	data-independent

7.5 Storage

A NeuralPSI server template is a 128-bit code - **16 bytes per user** - with no per-user key material, against Blind-Touch’s ~ 1.67 KB encrypted template (855 KB per 512 users) plus a one-time 117 MB Galois key (Table 5). The per-template reduction is $\sim 107\times$, and the 117 MB key is eliminated entirely; for a 512-user database the server-side totals are ~ 8 KB versus ~ 118 MB.

Table 5: Per-user server-side storage.

System	Per-user template	One-time key	512 users
Blind-Touch (HE/CKKS)	~ 1.67 KB	117 MB Galois key	~ 855 KB + 117 MB
NeuralPSI	16 B	~ 0	~ 8 KB

7.6 Ablation: Quantisation Dimension

We swept the Super-Bit code length $d \in \{64, 128, 256\}$ (i.e. $\{4, 8, 16\}$ orthonormalised 16×16 blocks), re-encoding the same held-out fingers under the same open-set protocol (Table 6). Accuracy *saturates* at $d = 128$: shrinking to 64 bits costs half a point of EER ($1.88\% \rightarrow 2.38\%$), while doubling to 256 bits buys nothing ($1.88\% \rightarrow 1.89\%$) yet doubles the template. This is the 16-dimensional information ceiling of §7.2 made concrete - 128 sign projections already capture essentially all of the ~ 16 bits of genuine information in the embedding, so the extra 128 projections at $d = 256$ are redundant.

Table 6: Quantisation-dimension ablation: EER and storage vs. Super-Bit code length d (224 model, plain Super-Bit, held-out identities). Accuracy saturates at $d = 128$.

d (bits)	Super-Bit blocks	storage (B/user)	EER	separation d'
64	4	8	2.38%	3.656
128	8	16	1.88%	3.778
256	16	32	1.89%	3.814

The trade-off therefore favours $d = 128$ on three counts. *Accuracy*: it is the knee - the smallest code that reaches the floating-point-limited ceiling. *Storage*: it grows linearly with d

(8/16/32 bytes/user), so 128 is half the footprint of 256 for equal accuracy. *Communication*: the online cost is dominated by the d -independent TLPSI term (§7.3), so it is essentially flat in d - d trades against storage and offline garbling, not online bandwidth. Finally, the reference FLPSI backend is built for $d = 128$ (a compiled 128-bit equality circuit and a 128-element OPRF key), making 128 the natively supported width.

8 Parameter Selection and Error Analysis

The FLPSI layer is parameterised by the code length d , the number of sub-samples T , the sub-sample width w , and the acceptance threshold θ . This section derives the error rates those parameters induce, and shows that the published choice $(d, T, w, \theta) = (128, 64, 14, 2)$ is wrong by three orders of magnitude on real codes — not because the arithmetic is wrong, but because the distribution it assumes is.

8.1 The accept model

A sub-sample selects w of the d positions uniformly at random and collides when the two codes agree on all of them. Writing $H = \|\tilde{\mathbf{x}} \oplus \tilde{\mathbf{y}}_i\|_1$ for the Hamming distance, the per-sub-sample collision probability and the acceptance probability are

$$q(H) = \frac{\binom{d-H}{w}}{\binom{d}{w}}, \quad \Pr[\text{accept} \mid H] = \Pr[\text{Binomial}(T, q(H)) \geq \theta]. \quad (5)$$

Acceptance depends on the pair *only through* H — not on which positions differ — because the mask is a uniformly random set. Consequently the false-accept and false-reject rates are exact functionals of the two Hamming histograms,

$$\text{FAR} = \sum_H P_{\text{imp}}(H) \Pr[\text{accept} \mid H], \quad \text{FRR} = 1 - \sum_H P_{\text{gen}}(H) \Pr[\text{accept} \mid H], \quad (6)$$

with no Monte-Carlo error. Every number below is computed from (6) over 1,200 held-out identities: 1,200 genuine and 2,877,600 impostor pairs.

Validating the model before using it. Equation (5) assumes the d positions are exchangeable, and Super-Bit’s advantage comes precisely from making bits *dependent*, so this needs testing. Measuring $\hat{q}(H)$ on real code pairs (200 sub-sample draws per pair, $w = 14$) gives $\hat{q}(H)/q(H)$ with **median** 1.002, **range** [0.919, 1.099] over $H \in [5, 40]$. Bit dependence is real — mean inter-bit $|\rho| = 0.254$, and the correlation matrix has participation ratio 9.84, so the 128 bits behave like about ten independent decisions — but it manifests in the distribution of H , not in $\Pr[\text{accept} \mid H]$. That isolates the defect to one place and licenses (6).

8.2 The codes are balanced but not uniform

The parameters were selected under a model in which impostor codes are uniform, giving $H \sim \text{Binomial}(128, \frac{1}{2})$ with $\sigma = 5.66$. Table 7 compares that model against measurement.

The mean is *exactly* right, because the quantiser’s median-thresholding step balances every bit (measured balance $\in [0.466, 0.540]$). **No first-moment check can detect this failure**: a sanity test verifying that impostor codes “look like coin flips on average” passes cleanly. The discrepancy is entirely in the second moment and the tail, where σ is $3.6\times$ larger than assumed.

The cause is structural rather than a defect of this particular bridge. A 128-bit Super-Bit code is 128 projections of a 16-dimensional embedding, so it carries roughly 16 real degrees of freedom. The bridge is *faithful* — $\mathbb{E}[H]/d = \arccos(\text{cos-sim})/\pi$ holds — and that faithfulness is the problem: it transmits the embedding’s angular distribution intact, including the fact

Table 7: Impostor Hamming distribution: assumed versus measured over 2,877,600 held-out pairs. The mean is exactly as predicted; nothing else is.

	uniform model	measured
mean H	64.0	64.0
standard deviation	5.66	20.22
minimum observed H	— (never below ≈ 40)	0
genuine mean H	—	8.3 (sd 5.67, max 34)

that some fingers genuinely resemble one another. We expect any LSH bridge over a learned low-dimensional embedding to behave the same way.

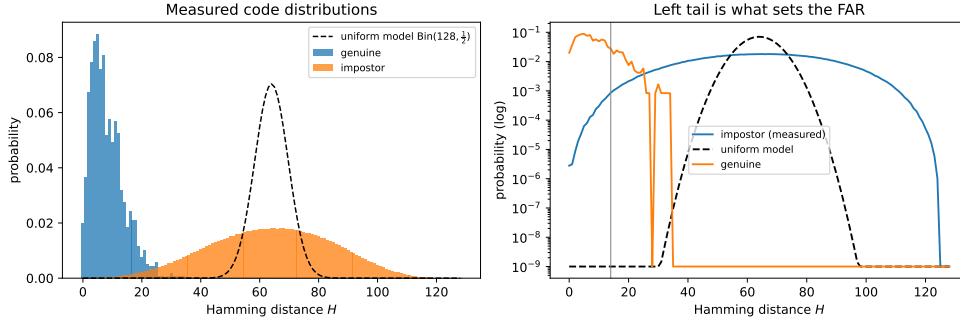


Figure 2: Measured genuine and impostor Hamming distributions against the uniform model (dashed). Left: linear scale, where the impostor mean sits exactly at $d/2$. Right: log scale, where the left tail — the part that sets the false-accept rate — is several orders of magnitude heavier than assumed.

8.3 The consequence for the published operating point

At $(w, \theta) = (14, 2)$ with error-free sub-samples, evaluating (6) against the uniform model reproduces the published per-record false-accept rate of 2.954×10^{-5} exactly, and $q(8) = \binom{120}{14} / \binom{128}{14} = 0.38493266$ matches the published constant to all quoted digits. The arithmetic is sound. Against the *measured* distribution the same expressions give

	uniform model	measured	ratio
per-record FAR	2.954×10^{-5}	4.578×10^{-2}	$1,550 \times$
FRR	—	9.61×10^{-3}	—

A per-record rate is not a security boundary. A $1:N$ identification query is tested against every enrolled record, so what an attacker faces is $\text{FAR}_{\text{query}} = 1 - (1 - \text{FAR})^N$ (Figure 3):

N	1	10	100	1,000	5,000	10^4
$\text{FAR}_{\text{query}}$, published point	0.046	0.374	0.991	1.000	1.000	1.000
$\text{FAR}_{\text{query}}$, collision floor	—	—	0.0003	0.0028	0.0138	0.0274

FRR is 0.96% throughout. **The published parameters describe a verification (1:1) system, not an identification system:** at $N = 1$ a 4.6% false-accept rate is poor but coherent, and by $N = 100$ the query-level rate already exceeds 50%. The second row is the irreducible floor derived in §8.5, which no parameter choice can go below.

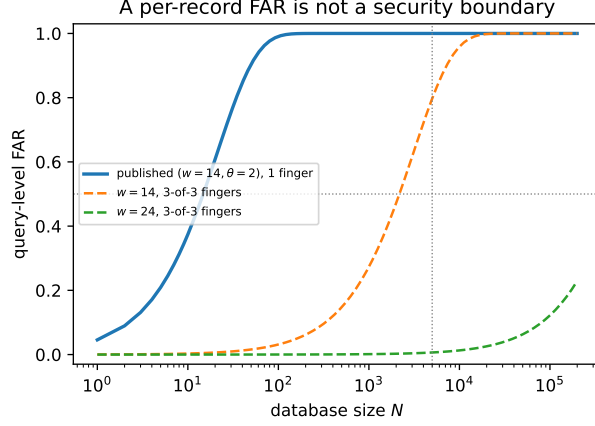


Figure 3: Query-level false-accept rate against database size. The published single-finger point crosses 50% at $N \approx 100$. Multi-finger fusion moves the curve by orders of magnitude; retuning does not.

8.4 Retuning does not repair it

We swept 6,848 combinations of $w \in \{8, \dots, 64\}$, $t \in \{0, \dots, 8\}$ and $\theta \in \{1, \dots, 64\}$, keeping only points whose FAR estimate is supported by the histogram (less than half the FAR mass drawn from bins holding fewer than ten observed pairs). Taking the lowest-FRR point that meets each query-level target at $N = 5,000$:

only one target is reachable at all. A query-level rate of 10^{-1} is met by $(w, t, \theta) = (56, 1, 40)$ — at an FRR of **84.8%** — and 10^{-2} , 10^{-3} and 10^{-4} are unreachable by any point in the sweep with usable histogram support. The only setting that reaches even a 10% query FAR rejects 85% of legitimate users. The suggestion that θ should grow as $\Theta(\log N)$ to hold the query-level rate constant is directionally right and quantitatively insufficient, for the reason we give next.

8.5 An irreducible collision floor

8 of the 2,877,600 impostor pairs have Hamming distance exactly zero — distinct fingers producing *identical* 128-bit codes. The collision rate is

$$p_0 = 2.78 \times 10^{-6}, \quad \text{exact Poisson 95\% CI } [1.20 \times 10^{-6}, 5.48 \times 10^{-6}]. \quad (7)$$

Since $q(0) = 1$, every sub-sample collides and $\Pr[\text{accept} \mid H=0] = 1$ for *every* (w, t, θ) . Therefore p_0 is a **hard lower bound on the per-record false-accept rate of any FLPSI parametrisation over these codes**, and it propagates to the query level as tabulated above. This is precisely why the ladder above becomes unreachable below 10^{-2} : at $N = 5,000$ the floor alone is 1.4%, independent of θ . The near tail is dense as well — 1.03×10^{-3} of impostor pairs lie at $H \leq 10$, and 2.80×10^{-2} at $H \leq 25$, which is the 99th percentile of the *genuine* distribution. The two distributions genuinely overlap; the protocol is being asked to separate things that are not separable.

The bound is a property of the code, not of the protocol. It moves only by producing better codes, and it is cheap for any future system to report: count $H = 0$ pairs on a held-out set. We recommend it as a standard diagnostic for fuzzy-PSI-over-learned-codes constructions.

8.6 Multi-finger fusion is the lever that works

Each finger is a separate enrolment, so k -of- K fusion multiplies evidence instead of trading FAR against FRR along one curve — and it is the only mechanism here that can beat the

single-finger floor, since k fingers must collide simultaneously. We measure it over 201 held-out subjects contributing at least three fingers each, computing a Poisson-binomial over the real per-finger accept probabilities, so any correlation between one person’s own fingers is carried by the data rather than assumed away.

Table 8: Measured k -of- K fusion at $N = 5,000$, over 201 held-out subjects.

parameters	rule	per-record FAR	FAR _{query}	FRR
$w=14, \theta=2$	1-of-3	1.31×10^{-1}	1.000	0.000
$w=14, \theta=2$	2-of-3	8.73×10^{-3}	1.000	0.001
$w=14, \theta=2$	3-of-3	3.19×10^{-4}	0.797	0.026
$w=24, \theta=2$	1-of-3	1.95×10^{-2}	1.000	0.002
$w=24, \theta=2$	2-of-3	2.45×10^{-4}	0.707	0.033
$w=24, \theta=2$	3-of-3	1.30×10^{-6}	0.0065	0.281

Fusion buys four orders of magnitude on the per-record rate, far more than any single-finger retuning achieves. It is still not sufficient on its own: the best measured configuration costs a 28% false-reject rate. The honest summary is that **at $N = 5,000$ with a 16-dimensional embedding, no configuration of this protocol is simultaneously secure and usable.**

What this means for parameter selection. Report FLPSI biometric parameters as a verification operating point unless the query-level rate at the target N is stated; fit them to the *measured* code distribution, since the uniform assumption errs by $1,550\times$ here and fails silently; and publish the collision floor of §8.5, which bounds every parameter choice and costs one pass over a held-out set. The remedy is more evidence, not a different threshold, and the binding constraint is the embedding dimension — sixteen dimensions cannot separate 5,000 identities at cryptographic error rates.

9 Related Work

Our work intersects four major research threads in secure computation and biometrics:

(1) HE-Based Biometric Authentication. Early work by Kim et al. [6] and Yang et al. [8] demonstrated that Homomorphic Encryption could be used for secure biometric matching. Systems like CryptoNets and, more recently, Blind-Touch pushed this boundary by integrating deep learning models directly with HE. While highly accurate, these systems fundamentally struggle with ciphertext expansion and the high computational cost of encrypted arithmetic.

(2) PSI and Fuzzy PSI. Private Set Intersection (PSI) allows two parties to compute the intersection of their sets without revealing anything else. Uzun et al. (2021) and Chakraborti et al. (2023) extended this to “fuzzy” matching, allowing items to match if they are close under a specific metric. The state-of-the-art Fuzzy-Labeled PSI (FLPSI) by Bui–Cong (2025) achieves linear scalability but requires inputs to already be in a binary Hamming space.

(3) Locality-Sensitive Hashing for Biometrics. Hashing techniques, such as BioHashing and various random projection codes, have long been explored to map continuous biometric features into discrete spaces. However, these techniques often suffer from significant accuracy degradation compared to modern CNN-based embeddings.

(4) Secure Computation for ML Inference. Frameworks like SecureML and XONN have explored replacing or modifying neural network architectures to make them more amenable to secure multiparty computation, often by binarizing weights or activations.

NeuralPSI is uniquely positioned as the first system to bridge the gap between HE-level perceptual accuracy (Thread 1) and FLPSI-level communication efficiency (Thread 2), using a specialized LSH quantization bridge (Thread 3) to strictly decouple feature extraction from secure inference (Thread 4).

10 Discussion

Accuracy Trade-Off. The +1.55 percentage-point increase in EER ($0.33\% \rightarrow 1.88\%$) is the direct price paid for binary quantization. While acceptable for many applications, future work could explore ternary or product quantization to recover this fractional accuracy loss without entirely abandoning the efficient Hamming-based protocol framework.

Plaintext Template Storage. Unlike Blind-Touch, NeuralPSI stores the binary templates in plaintext on the server to enable the fast FLPSI scan. While the 128-bit strings lack direct biometric semantics, an adversary possessing both the public projection matrix $\mathbf{P}$ and the template $\tilde{\mathbf{y}}$ might attempt to approximate the original embedding direction. Future iterations could mitigate this by storing encrypted templates combined with an oblivious lookup protocol, trading some offline storage efficiency for stronger data-at-rest guarantees.

Semi-Honest Limitation. Both NeuralPSI and the Blind-Touch baseline target semi-honest adversaries. Real-world deployments may require security against malicious actors who deviate from the protocol. Upgrading NeuralPSI to malicious security is future work. Cut-and-choose techniques exist for related structure-aware PSI settings [5], but those constructions target geometric-ball predicates rather than the threshold-PSI functionality used here, so their overheads do not transfer directly and we do not estimate them.

11 Conclusion

Privacy-preserving cloud fingerprint authentication has historically forced a compromise: systems either achieve CNN-level accuracy at the cost of massive computational and communication overhead (via Homomorphic Encryption), or they achieve high scalability but fail to incorporate robust perceptual feature extraction.

NeuralPSI demonstrates that CNN feature extraction and FLPSI-based matching are highly complementary. By recognizing that the CNN can handle the hard perceptual problem in plaintext on the client, and FLPSI can handle the strict privacy problem on the server, we decouple the two responsibilities. This decoupling is the key design principle of the system.

Using a public, 128-bit Super-Bit quantization bridge, we achieve a near-identical accuracy profile to the heavily-encrypted Blind-Touch system (1.88% EER), while completely eliminating the 117 MB Galois key overhead. Furthermore, NeuralPSI reduces per-user server-side template storage by $\approx 107\times$ (1.67 KB to 16 bytes). Its matching cost is linear in the database size with no slot-packing ceiling; we measure this to $N = 5 \times 10^4$ and characterise it analytically beyond that.

Ultimately, this decoupling principle generalizes well beyond fingerprint authentication. By bridging continuous neural embeddings into balanced Hamming spaces, this architecture can securely scale face, iris, and voice biometrics anywhere a CNN produces a compact feature vector.

References

- [1] ISO/IEC 24745:2022 information security, cybersecurity and privacy protection – biometric information protection, 2022.
- [2] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In *Advances in Cryptology – CRYPTO*, 2019.
- [3] Dung Bui and Kelong Cong. Efficient fuzzy labeled PSI from vector ring-OLE. In *Cryptology and Network Security (CANS 2025)*, volume 16351 of *Lecture Notes in Computer Science*. Springer, 2026. Cryptology ePrint Archive, Paper 2025/1470.

- [4] Hyunmin Choi, Simon S. Woo, and Hyoungshick Kim. Blind-touch: Homomorphic encryption-based distributed neural network inference for privacy-preserving fingerprint authentication. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 21976–21985, 2024.
- [5] Gayathri Garimella, Mike Rosulek, and Jaspal Singh. Malicious secure, structure-aware private set intersection. In *Advances in Cryptology – CRYPTO, 2023*.
- [6] Taeyun Kim, Yousung Oh, and Hyoungshick Kim. Efficient privacy-preserving fingerprint-based authentication system using fully homomorphic encryption. *Security and Communication Networks*, pages 1–11, 2020.
- [7] Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In *Advances in Cryptology – EUROCRYPT 2021*, Lecture Notes in Computer Science, pages 901–930, 2021.
- [8] Wencheng Yang, Song Wang, Kan Yu, James Jin Kang, and Michael N. Johnstone. Secure fingerprint authentication with homomorphic encryption. In *Digital Image Computing: Techniques and Applications (DICTA)*, pages 1–6, 2020.

A Full Hybrid Sequence

This appendix gives the two simulators of §5 in full and expands §5.4 with the distinguisher reductions. Throughout, $\mathcal{D}$ is a distinguisher, view_i its view in H_i , and $\varepsilon_i = |\Pr[\mathcal{D}(\text{view}_{i-1}) = 1] - \Pr[\mathcal{D}(\text{view}_i) = 1]|$.

Simulators

Simulator for a corrupt client. Sim_{C} receives $\tilde{\mathbf{x}}$ together with $(\text{out}, \mathcal{L}_{\mathsf{C}})$ and must produce the mOPRF outputs, the sender-side silent-VOLE transcript, and the polynomials $\{\hat{z}_i\}_{i \in [N]}$.

1. Sample $r_j \leftarrow \mathcal{D}$ uniformly for $j \in [T]$ as the mOPRF outputs and set $\hat{X}'' = \{r_j \oplus j\}_{j \in [T]}$.
2. For each $i \in [N]$: if $i \in \text{out}$, sample $\hat{\ell}'_i \leftarrow \mathbb{F}$ and a uniform polynomial $\hat{f}_i$ of degree $\leq \theta - 1$ with $\hat{f}_i(0) = \hat{\ell}'_i$; otherwise set $\hat{f}_i = \perp$.
3. Build $\hat{z}_i$ by fixing its values on the $2T + 1$ interpolation points: at points indexed by $j \in S_i$ (read from $\mathcal{L}_{\mathsf{C}}$) set $\hat{z}_i(x_j) = \hat{f}_i(x_j)$; at every other point sample uniformly from $\mathbb{F}$; interpolate the unique polynomial of degree $\leq 2T$.
4. Emit the sender’s correlation messages as uniform vectors of the appropriate length.

Simulator for a corrupt server. Sim_{S} receives D and the PRF key k and outputs $\perp$, so its only task is the transcript. The single input-dependent message is $\Delta U = \Delta u(X)$ with $\Delta u = u_0 - u'_0$. It is worth stating precisely why this is simulatable, because the natural argument is slightly wrong: ΔU is *not* uniform on $\mathbb{F}^{2T+1}$. It lies in the $(T+1)$ -dimensional image of the evaluation map on polynomials of degree $\leq T$. The simulator therefore samples a uniform polynomial of degree $\leq T$ and outputs its evaluation vector, and simulates the silent-VOLE receiver messages with the base-VOLE simulator. Since u'_0 is uniform, $u_0 - u'_0$ is uniform in *that subspace*, and the indistinguishability is statistical rather than computational.

Hybrid reductions

$H_0 \rightarrow H_1$ (**mOPRF**). Replace the masked-OPRF sub-protocol with $\mathcal{F}_{\text{mOPRF}}^T$. Given $\mathcal{D}$ distinguishing these hybrids, build $\mathcal{B}$ against semi-honest mOPRF security: $\mathcal{B}$ runs the rest of the protocol honestly, forwards its challenge transcript in place of the sub-protocol messages, and outputs $\mathcal{D}$'s guess. Hence $\varepsilon_1 \leq \varepsilon_{\text{mOPRF}}$.

$H_1 \rightarrow H_2$ (**PRF**). Replace F_k by a uniformly random function. The reduction $\mathcal{B}$ queries its oracle wherever the protocol evaluates F_k . The query budget is the subtlety: each mask restricts the PRF input to a domain of size 2^w , so a distinguisher may enumerate the entire input table of $T \cdot 2^w$ entries, and $\mathcal{B}$ must be allowed that many oracle calls. With $w = 14$ and $T = 64$ this is 2^{20} , giving $\varepsilon_2 \leq \varepsilon_F(q + T \cdot 2^w)$. Omitting this factor is the most common way to understate the bound for this construction.

$H_2 \rightarrow H_3$ (**VOLE**). Replace the vector ring-OLE sub-protocol with $\mathcal{F}_{\text{VOLE}}$. Indistinguishability follows from base-VOLE security and the LPN assumption used by the silent extension [2, 7], so $\varepsilon_3 \leq \varepsilon_{\text{VOLE}} + \varepsilon_{\text{LPN}}$.

$H_3 \rightarrow H_4$ (**statistical masking**). For every non-colliding evaluation point x_j , replace $v_1(x_j) = f(x_j) + (\prod_i (x_j - \tilde{y}_i)) \cdot R(x_j)$ by a uniform element of $\mathbb{F}$.

Fix a record i and let $J = \{j \in [T] : j \notin S_i\}$ be its non-colliding points. At each such point $\prod_i (x_j - \tilde{y}_i) \neq 0$, so the map $R \mapsto (\prod_i (x_j - \tilde{y}_i) R(x_j))_{j \in J}$ is a scaled evaluation map. Since R is uniform of degree $\leq T$, its evaluation vector on any set of at most $T + 1$ distinct points is uniform on $\mathbb{F}^{|J|}$ — the evaluation map on $\deg R + 1$ points is a bijection onto $\mathbb{F}^{\deg R + 1}$. The masking terms are therefore jointly uniform, and the substitution is perfect, *provided*

$$|J| \leq \deg R + 1 = T + 1. \quad (8)$$

Here $|J| \leq T$ always, so (8) holds unconditionally — but it is a design constraint, not an accident: it fails if $\deg R$ is reduced below T while tuning the sub-sample count downward to save communication.

Applying this simultaneously at all N records and all $2T + 1$ points, the union bound over the event that any evaluation point coincides with a root gives $\varepsilon_4 \leq N(2T + 1)/|\mathbb{F}|$, which is the statistical term of (3) and the reason $|\mathbb{F}|$ must be chosen with N in view.

$H_4 \rightarrow H_5$ (**labels**). Replace $\ell'_i = F_k(\ell_i)$ by uniform field elements. Immediate from H_2 , in which F_k is already a random function; $\varepsilon_5 = 0$.

$H_5 = \text{Sim}_{\mathcal{C}}$. In H_5 every message the client sees is either uniform or determined by $(\tilde{\mathbf{x}}, \text{out}, \mathcal{L}_{\mathcal{C}})$, which is exactly the simulator's input, so the two distributions are identical by inspection.

Composition across N records. All N record instances share one receiver polynomial u_0 and one PRF key k , across records and sessions. This is why H_2 and H_4 are stated over the whole database rather than per record: H_2 replaces the shared F_k once, so its cost is charged once rather than N times, and H_4 is applied simultaneously at all $N(2T + 1)$ points under a single union bound. A per-record invocation of a single-instance theorem would not establish this, because correlated reuse across instances is precisely where such compositions fail.

Summing the terms gives (3).